

# CUDA Kernel 面试背题笔记

---

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

**30 个 Kernel · 9 个 Phase · 全中文注释**

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 8 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cstring>
119 #include <cuda_fp16.h>
120 #include <cuda_runtime.h>
121 #include <float.h>
122 #include <stdio.h>
123 #include <stdlib.h>
124 #include <math.h>
125 #include <cublas_v2.h>

```

```

126 #include <cuda.h>
127 #include <cuda/barrier>
128
129 #define WARP_SIZE 32
130 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
131 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
132 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
133
134 // =====
135 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
136 // =====
137
138 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
139 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
140 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
141 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
142 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
143 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
144 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
145 //
146 // 初始: 每个 lane 持有自己的值 v0..v7
147 // lane:  0    1    2    3    4    5    6    7
148 // val:  v0   v1   v2   v3   v4   v5   v6   v7
149 //
150 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
151 //
152 //      ┌──────────┐
153 // 对:   (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane:  0    1    2    3    4    5    6    7
156 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //
160 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
161 // 对:   (0,2) (1,3) (4,6) (5,7)
162 //      ─── 前4个一组 ───      ─── 后4个一组 ───
163 //
164 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
165 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
166 //      = v0+v2+v4+v6 ... (逐步归约)
167 //
168 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
169 //
170 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
171 // 对:   (0,1) (2,3) (4,5) (6,7)
172 //
173 // lane:  0    1    2    3    4    5    6    7
174 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
175 //
176 // mask=0: 循环终止, 归约完成。
177 //
178 // 关键性质:
179 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
180 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
181 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
182 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
183 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
184 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
185
186 template <const int kWarpSize = WARP_SIZE, typename T = float>
187 __device__ __forceinline__ T warp_reduce_sum(T val) {
188 #pragma unroll
189     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
190         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
191     }
192     return val;
193 }

```

```

189 }
190
191 // Warp Reduce Max — generic
192 template <const int kWarpSize = WARP_SIZE, typename T = float>
193 __device__ __forceinline__ T warp_reduce_max(T val) {
194 #pragma unroll
195     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
196         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
197     }
198     return val;
199 }
200
201 // =====
202 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
203 // =====
204
205 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
206 // 两级归约流程:
207 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
208 // 2. warp leader (lane=0) 写入 shared memory
209 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
210 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
211 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
212 template <const int NUM_THREADS = 256>
213 __device__ float block_reduce_sum(float val) {
214     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
215     int warp = threadIdx.x / WARP_SIZE;
216     int lane = threadIdx.x % WARP_SIZE;
217     __shared__ float shared[NUM_WARPS];
218
219     float value = warp_reduce_sum<WARP_SIZE>(val);
220     if (lane == 0)
221         shared[warp] = value;
222     __syncthreads();
223     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
224     value = warp_reduce_sum<NUM_WARPS>(value);
225     // 关键: broadcast 结果到所有线程, 后续用 result
226     // 做除法等操作时所有线程都能拿到
227     value = __shfl_sync(0xffffffff, value, 0, 32);
228     return value;
229 }
230
231 // Block Reduce Max — FP32 (增强版, 带 broadcast)
232 template <const int NUM_THREADS = 256>
233 __device__ float block_reduce_max(float val) {
234     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
235     int warp = threadIdx.x / WARP_SIZE;
236     int lane = threadIdx.x % WARP_SIZE;
237     __shared__ float shared[NUM_WARPS];
238
239     float value = warp_reduce_max<WARP_SIZE>(val);
240     if (lane == 0)
241         shared[warp] = value;
242     __syncthreads();
243     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
244     value = warp_reduce_max<NUM_WARPS>(value);
245     value = __shfl_sync(0xffffffff, value, 0, 32);
246     return value;
247 }
248
249 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
250 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
251 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

252 // Grid: ((N + 255) / 256, 1, 1)
253 // Block: (256, 1, 1)
254 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
255 template <const int NUM_THREADS = 256>
256 __global__ void block_reduce_all(float *a, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * NUM_THREADS + tid;
259     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
260     __shared__ float shared[NUM_WARPS];
261
262     float val = (idx < N) ? a[idx] : 0.0f;
263     int warp = tid / WARP_SIZE;
264     int lane = tid % WARP_SIZE;
265
266     val = warp_reduce_sum<WARP_SIZE>(val);
267     if (lane == 0)
268         shared[warp] = val;
269     __syncthreads();
270
271     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
272     if (warp == 0)
273         val = warp_reduce_sum<NUM_WARPS>(val);
274     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
275         atomicAdd(y, val);
276 }
277
278 // ---- Dot Product: y = sum(a[i] * b[i]) ----
279 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
280 // Grid: ((N + 255) / 256, 1, 1)
281 // Block: (256, 1, 1)
282 // source: LeetCUDA/kernels/dot-product/dot_product.cu
283 template <const int NUM_THREADS = 256>
284 __global__ void dot(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = blockIdx.x * NUM_THREADS + tid;
287     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
288     __shared__ float shared[NUM_WARPS];
289
290     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
291     int warp = tid / WARP_SIZE;
292     int lane = tid % WARP_SIZE;
293
294     prod = warp_reduce_sum<WARP_SIZE>(prod);
295     if (lane == 0)
296         shared[warp] = prod;
297     __syncthreads();
298
299     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
300     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
301         prod = warp_reduce_sum<NUM_WARPS>(prod);
302     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
303         atomicAdd(y, prod);
304 }
305
306 // Dot Product + float4
307 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
308 // Block: (64, 1, 1), 256/4=64
309 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
310 // source: LeetCUDA/kernels/dot-product/dot_product.cu
311 template <const int NUM_THREADS = 256 / 4>
312 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
313     int tid = threadIdx.x;
314     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;

```

```

315 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
316 __shared__ float shared[NUM_WARPS];
317
318 float4 reg_a = FLOAT4(a[idx]);
319 float4 reg_b = FLOAT4(b[idx]);
320 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
321                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
322                  : 0.0f;
323 int warp = tid / WARP_SIZE;
324 int lane = tid % WARP_SIZE;
325
326 prod = warp_reduce_sum<WARP_SIZE>(prod);
327 if (lane == 0)
328     shared[warp] = prod;
329 __syncthreads();
330
331 prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
332 if (warp == 0)
333     prod = warp_reduce_sum<NUM_WARPS>(prod);
334 if (tid == 0)
335     atomicAdd(y, prod);
336 }
337
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCUDA/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);
370         reg_y.z = fmaxf(0.0f, reg_x.z);
371         reg_y.w = fmaxf(0.0f, reg_x.w);
372         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373     }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)

```

```

378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&(y[a[idx]]), 1);
418 }
419
420 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
421 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
422 //
423 // 面试要点:
424 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
425 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
426 // - 数学公式 (5 步推导):
427 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
428 //       L_max = max(LSE_1, LSE_2)
429 //   Step 2 — 还原指数权重 (un-normalized):
430 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
431 //   Step 3 — 归一化为混合比例:
432 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
433 //       满足 alpha + beta = 1
434 //   Step 4 — 逐元素加权合并:
435 //       O = alpha * O_1 + beta * O_2
436 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
437 //   lse[head_idx][token_idx])
438 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
439 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
440 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score

```



```

441 // 为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
442 // 该段权重退化为 0
443 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
444 // pack load → FMA → pack store 一条流水线
445 // -  $\text{AI} \approx 3 \text{ FLOP} / 20 \text{ bytes} \approx 0.15 \text{ FLOPS/Byte} \rightarrow \text{severely memory-bound}$ 
446 //
447 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
448 // pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
449 // threads_per_head = head_size / 4 = 2
450 // total_threads = num_tokens * num_heads * threads_per_head = 8
451 //
452 // global_idx:      0      1      2      3      4      5      6      7
453 // token_head_idx:  0      0      1      1      2      2      3      3
454 // (第几个 (token,head) 对, 行优先)
455 // pack_idx:        0      1      0      1      0      1      0      1
456 // (该 (token,head) 对内第几个 pack)
457 // token_idx:       0      0      0      0      1      1      1      1
458 // (token_head_idx / num_heads, token 变化慢)
459 // head_idx:        0      0      1      1      0      0      1      1
460 // (token_head_idx % num_heads, head 变化快)
461 // pack_offset:     0      4      0      4      0      4      0      4
462 // (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
463 // head_offset:     0      0      8      8      16     16     24     24
464 // (token_idx * num_heads * head_size + head_idx * head_size,
465 // 该 (token,head) 对在 output 展平数组中的起始偏移)
466 //
467 // Grid: ((total_threads + 127) / 128, 1, 1)
468 // Block: (128, 1, 1)
469 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
470 __global__ void merge_attn_states(
471     float *output,           // [num_tokens, num_heads, head_size]
472     const float *prefix_output, // [num_tokens, num_heads, head_size]
473     const float *prefix_lse,   // [num_heads, num_tokens]
474     const float *suffix_output, // [num_tokens, num_heads, head_size]
475     const float *suffix_lse,   // [num_heads, num_tokens]
476     int num_tokens, int num_heads, int head_size) {
477
478     constexpr int NUM_THREADS = 128;
479     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
480     using pack_t = uint4;
481
482     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
483     const int threads_per_head = head_size / PACK_SIZE;
484     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
485     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
486     const int total_threads = num_tokens * num_heads * threads_per_head;
487
488     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
489     if (global_idx >= total_threads)
490         return;
491
492     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
493     // token_head_idx: 第几个 (token, head) 对, 行优先展平
494     const int token_head_idx = global_idx / threads_per_head;
495     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
496     const int pack_idx = global_idx % threads_per_head;
497
498     // token_head_idx 分解为 (token_idx, head_idx)
499     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
500     const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
501
502     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
503     const int pack_offset = pack_idx * PACK_SIZE;

```

```

504 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
505 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
506
507 // 定位到当前 (token, head) 的输出段起始
508 const float *prefix_head = prefix_output + head_offset;
509 const float *suffix_head = suffix_output + head_offset;
510 float *output_head = output + head_offset;
511
512 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
513 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
514 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
515
516 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
517 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
518 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
519
520 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
521 const float max_lse = fmaxf(p_lse, s_lse);
522 p_lse -= max_lse;
523 s_lse -= max_lse;
524
525 // Step 2-3: 指数还原 → 混合比例 alpha, beta
526 const float p_se = expf(p_lse);
527 const float s_se = expf(s_lse);
528 const float out_se = p_se + s_se;
529 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
530 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
531
532 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
533 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
534 if (pack_offset < head_size) {
535     pack_t p_pack =
536         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
537     pack_t s_pack =
538         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
539     pack_t o_pack;
540
541 #pragma unroll
542     for (int i = 0; i < PACK_SIZE; ++i) {
543         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
544         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
545         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
546         reinterpret_cast<float *>(&o_pack)[i] = o_v;
547     }
548
549     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
550 }
551 }
552
553 // =====
554 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
555 // =====
556 // 面试要点:
557 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
558 //     online (1-pass, 增量更新)
559 //   - Online Softmax 是 FlashAttention 的数学基础
560 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
561 //     + variance)
562 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
563
564 // =====
565 // Phase 3a: Softmax — 三级递进 (面试核心考点)
566 // =====

```

```

567 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
568 // 回答线索: naive(溢出) → safe → online
569
570 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
571 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
572 // 问题: x 值很大时 exp(x) 溢出为 inf
573 template <const int NUM_THREADS = 256>
574 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
575 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
576 // source: LeetCUDA/kernels/softmax/softmax.cu
577 __global__ void softmax_per_token(float *x, float *y, int N) {
578     const int tid = threadIdx.x;
579     const int idx = blockIdx.x * blockDim.x + tid;
580
581     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
582     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
583     if (idx < N)
584         y[idx] = exp_val / exp_sum;
585 }
586
587 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
588 // 面试重点: 为什么 Softmax 需要 Safe?
589 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
590 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
591 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
592 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
593 // Grid: (S, 1, 1)
594 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
595 // source: LeetCUDA/kernels/softmax/softmax.cu
596 template <const int NUM_THREADS = 256>
597 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
598     const int tid = threadIdx.x;
599     const int idx = blockIdx.x * blockDim.x + tid;
600
601     // Pass 1: block reduce max — 找最大值
602     float val = (idx < N) ? x[idx] : -FLT_MAX;
603     float max_val = block_reduce_max<NUM_THREADS>(val);
604
605     // Pass 2: exp(x - max) → block reduce sum
606     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
607     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
608
609     if (idx < N)
610         y[idx] = exp_val / exp_sum;
611 }
612
613 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
614 // 面试重点:
615 // 两种使用场景 (公式不同, 但结果等价):
616 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
617 //       m_new = max(m_old, x_i)
618 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
619 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
620 //       m = max(m1, m2) safe softmax用的max值
621 //       d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
622 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
623 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
624 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
625 //
626 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
627 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
628 //   (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
629 //   (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2

```

```

630 //
631 // 合并公式（对称，不依赖"新/旧"概念）：
632 //   m = max(m1, m2)
633 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
634 //
635 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
636 struct __align__(8) MD {
637     float m; // running max
638     float d; // running denominator (sum of exp(x - max))
639 };
640
641 template <const int kWarpSize = WARP_SIZE>
642 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
643     #pragma unroll
644     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
645         MD md2;
646         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
647         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
648
649         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
650         MD s_m = (md1.m > md2.m) ? md2 : md1;
651
652         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
653         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
654         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
655         md1.m = b_m.m;
656     }
657     return md1;
658 }
659
660 // 注意：这里默认一个 block 处理一个 token；边界线程的 d=0 只参与归约，不会写回 y
661 // Grid: (S, 1, 1)
662 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
663 // source: LeetCUDA/kernels/softmax/softmax.cu
664 template <const int NUM_THREADS = 256>
665 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
666     int tid = threadIdx.x;
667     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
668     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
669     __shared__ MD shared[NUM_WARPS];
670
671     float val = (idx < N) ? x[idx] : -FLT_MAX;
672     int warp = tid / WARP_SIZE;
673     int lane = tid % WARP_SIZE;
674
675     // 初始化：每个线程持有一个 (max, denom) 对
676     MD md;
677     md.m = idx < N ? val : -FLT_MAX;
678     md.d = idx < N ? 1.0f : 0.0f;
679
680     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
681     md = warp_reduce_md<WARP_SIZE>(md);
682
683     if (lane == 0)
684         shared[warp] = md;
685     __syncthreads();
686
687     // 第二级归约：每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
688     md = lane < NUM_WARPS ? shared[lane] : MD{-FLT_MAX, 0.0f};
689     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
690
691     // 用全局 max 和 denom 做最终 softmax
692     float d_inv = __fdividef(1.0f, md.d);

```

```

693 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
694 if (idx < N) {
695     y[idx] = __expf(val - md.m) * d_inv;
696 }
697 }
698
699 // =====
700 // Phase 3b: RMS Normalization (1-pass reduce)
701 // =====
702 // 面试要点:
703 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
704 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
705 //   - Llama 系列使用 RMS Norm
706 //   - grid(N, K/K), block(K): 一行一个 block
707 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
708 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
709 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
710 template <const int NUM_THREADS = 128>
711 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
712     int tid = threadIdx.x;
713     int idx = blockIdx.x * blockDim.x + threadIdx.x;
714     const float epsilon = 1e-5f;
715
716     __shared__ float s_variance;
717     float value = (idx < N * K) ? x[idx] : 0.0f;
718     float variance = value * value;
719     variance = block_reduce_sum<NUM_THREADS>(variance);
720     if (tid == 0)
721         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
722     __syncthreads();
723     if (idx < N * K)
724         y[idx] = (value * s_variance) * g;
725 }
726
727 // RMS Norm + float4
728 // Grid: (N, 1, 1)
729 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
730 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
731 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
732 template <const int NUM_THREADS = 128 / 4>
733 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
734     int tid = threadIdx.x;
735     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
736     const float epsilon = 1e-5f;
737
738     __shared__ float s_variance;
739     float4 reg_x = FLOAT4(x[idx]);
740     float4 variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
741                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
742                                     : 0.0f;
743     variance = block_reduce_sum<NUM_THREADS>(variance);
744     if (tid == 0)
745         s_variance = rsqrtf(variance / (float)K + epsilon);
746     __syncthreads();
747     float4 reg_y;
748     reg_y.x = reg_x.x * s_variance * g;
749     reg_y.y = reg_x.y * s_variance * g;
750     reg_y.z = reg_x.z * s_variance * g;
751     reg_y.w = reg_x.w * s_variance * g;
752     if (idx < N * K)
753         FLOAT4(y[idx]) = reg_y;
754 }
755

```

```

756 // =====
757 // Phase 3c: Layer Normalization (2-pass reduce)
758 // =====
759 // 面试要点:
760 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
761 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
762 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
763 // Grid: (N, 1, 1), 一行一个 block
764 // Block: (128, 1, 1), NUM_THREADS=K=128
765 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
766 template <const int NUM_THREADS = 128>
767 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
768     int tid = threadIdx.x;
769     int idx = blockIdx.x * blockDim.x + threadIdx.x;
770     const float epsilon = 1e-5f;
771
772     __shared__ float s_mean;
773     __shared__ float s_variance;
774     float value = (idx < N * K) ? x[idx] : 0.0f;
775
776     // Pass 1: compute mean
777     float sum = block_reduce_sum<NUM_THREADS>(value);
778     if (tid == 0)
779         s_mean = sum / (float)K;
780     __syncthreads(); // 必须等待 s_mean 对所有线程可见
781
782     // Pass 2: compute variance = (x - mean)2
783     float variance = (value - s_mean) * (value - s_mean);
784     variance = block_reduce_sum<NUM_THREADS>(variance);
785     if (tid == 0)
786         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
787     __syncthreads(); // 必须等待 s_variance 对所有线程可见
788
789     if (idx < N * K)
790         y[idx] = ((value - s_mean) * s_variance) * g + b;
791 }
792
793 // Layer Norm + float4
794 // Grid: (N, 1, 1)
795 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
796 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
797 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
798 template <const int NUM_THREADS = 128 / 4>
799 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
800     int tid = threadIdx.x;
801     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
802     const float epsilon = 1e-5f;
803
804     __shared__ float s_mean;
805     __shared__ float s_variance;
806     float4 reg_x = FLOAT4(x[idx]);
807     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
808
809     float sum = block_reduce_sum<NUM_THREADS>(value);
810     if (tid == 0)
811         s_mean = sum / (float)K;
812     __syncthreads();
813
814     float4 reg_x_hat;
815     reg_x_hat.x = reg_x.x - s_mean;
816     reg_x_hat.y = reg_x.y - s_mean;
817     reg_x_hat.z = reg_x.z - s_mean;
818     reg_x_hat.w = reg_x.w - s_mean;

```

```

819 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
820         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
821 variance = block_reduce_sum<NUM_THREADS>(variance);
822 if (tid == 0)
823     s_variance = rsqrtf(variance / (float)K + epsilon);
824 __syncthreads();
825
826 float4 reg_y;
827 reg_y.x = reg_x_hat.x * s_variance * g + b;
828 reg_y.y = reg_x_hat.y * s_variance * g + b;
829 reg_y.z = reg_x_hat.z * s_variance * g + b;
830 reg_y.w = reg_x_hat.w * s_variance * g + b;
831 if (idx < N * K)
832     FLOAT4(y[idx]) = reg_y;
833 }
834
835 // =====
836 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
837 // =====
838 // 面试要点:
839 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
840 //     [x1']   [cos(θ)  -sin(θ)] [x1]
841 //     [x2'] = [sin(θ)   cos(θ)] [x2]
842 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
843 //   - token_pos = idx / N: token 在序列中的位置
844 //   - token_idx = idx % N: token 内的维度对索引
845 //   - 输入 [seq_len, hidden_size], 输出同形状
846
847 // Grid: ((seq_len * N + 255) / 256, 1, 1)
848 // Block: (256, 1, 1)
849 // source: LeetCUDA/kernels/rope/rope.cu
850 __global__ void rope(float *x, float *out, int seq_len, int N) {
851     int idx = blockIdx.x * blockDim.x + threadIdx.x;
852     float x1 = x[idx * 2];
853     float x2 = x[idx * 2 + 1];
854     int token_pos = idx / N; // 序列位置
855     int token_idx = idx % N; // 维度对索引
856
857     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
858     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
859     float sin_v = sinf(token_pos * exp_v);
860     float cos_v = cosf(token_pos * exp_v);
861
862     // 2D 旋转
863     float out1 = x1 * cos_v - x2 * sin_v;
864     float out2 = x1 * sin_v + x2 * cos_v;
865     out[idx * 2] = out1;
866     out[idx * 2 + 1] = out2;
867 }
868
869 // =====
870 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
871 // =====
872 // 面试要点 (Bank Conflict 专题):
873 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
874 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
875 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
876 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
877 //
878 // 转置的四步演进:
879 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
880 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
881 //   BCF:   smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict

```



```

882 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
883 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
884
885 // 每个线程处理 1 个元素, block(16,16)
886 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
887 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
888 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
889 // Block: (16, 16, 1)
890 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
891 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
892     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
893     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
894     if (r < row && c < col) {
895         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
896         y[c * row + r] = x[r * col + c];
897     }
898 }
899
900 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
901 // Block: (16, 16, 1)
902 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
903 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
904 __global__ void mat_transpose_padded(
905     float *x, float *y, const int row, const int col) {
906     const int tx = threadIdx.x, ty = threadIdx.y;
907
908     constexpr int TILE = 16;
909     constexpr int PAD = 1;
910     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
911     __shared__ float tile[TILE * 4][TILE + PAD];
912
913     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
914     const int x_c = blockIdx.x * TILE + tx;
915     const int x_r = blockIdx.y * TILE + ty;
916
917     if (x_r * 4 < row && x_c < col) {
918         // Step 1: 4 rows × 1 col → smem (row-major);
919 #pragma unroll
920         for (int i = 0; i < 4; ++i) {
921             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
922         }
923         __syncthreads();
924
925         // Step 2: Transposed read from smem
926         float4 reg_trans;
927         reg_trans.x = tile[tx * 4 + 0][ty];
928         reg_trans.y = tile[tx * 4 + 1][ty];
929         reg_trans.z = tile[tx * 4 + 2][ty];
930         reg_trans.w = tile[tx * 4 + 3][ty];
931
932         // y 空间坐标: y_r = x 的列, y_c = x 的行
933         const int y_r = blockIdx.x * TILE + ty; // = x col
934         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
935
936         // Coalesced write: y[y_r][y_c .. y_c+3]
937         FLOAT4(y[y_r * row + y_c]) = reg_trans;
938     }
939 }
940
941 // =====
942 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
943 // =====
944 // 面试要点:

```



```

945 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
946 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
947 // - 不同 K 值对应不同分块策略:
948 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
949 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
950 //   K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
951 //
952 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
953
954 // ---- SGEMV K32: 基础 warp-per-row ----
955 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
956 // grid(M/4), 每个 warp 负责一行
957 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
958 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
959 // Block: (32, 4, 1), 每行 1 warp
960 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
961 // source: LeetCUDA/kernels/sgemv/sgemv.cu
962 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
963     int tx = threadIdx.x; // 0~31
964     int ty = threadIdx.y; // 0~3
965     int lane = tx % WARP_SIZE; // 0~31
966     int m = blockIdx.x * blockDim.y + ty; // 全局行号
967     if (m < M) {
968         float sum = 0.0f;
969         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
970         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
971         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
972 #pragma unroll
973         for (int w = 0; w < NUM_ITERS; ++w) {
974             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
975             int k = w * WARP_SIZE + lane;
976             sum += a[m * K + k] * x[k];
977         }
978         sum = warp_reduce_sum<WARP_SIZE>(sum);
979         // 每个 warp 处理一行, lane 0 写回结果
980         if (lane == 0)
981             y[m] = sum;
982     }
983 }
984
985 // ---- SGEMV K128: float4 向量化 ----
986 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
987 // Grid: ((M + 3) / 4, 1, 1)
988 // Block: (32, 4, 1)
989 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
990 // source: LeetCUDA/kernels/sgemv/sgemv.cu
991 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
992     int tx = threadIdx.x; // 0~31
993     int ty = threadIdx.y; // 0~3
994     int lane = tx % WARP_SIZE; // 0~31
995     int m = blockDim.y * blockIdx.x + ty;
996
997     if (m < M) {
998         float sum = 0.0f;
999         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1000         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1001 #pragma unroll
1002         for (int w = 0; w < NUM_ITERS; ++w) {
1003             int k = (w * WARP_SIZE + lane) * 4;
1004             float4 reg_x = FLOAT4(x[k]);
1005             float4 reg_a = FLOAT4(a[m * K + k]);
1006             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1007                    reg_a.w * reg_x.w);

```

```

1008     }
1009     sum = warp_reduce_sum<WARP_SIZE>(sum);
1010     if (lane == 0)
1011         y[m] = sum;
1012 }
1013 }
1014
1015 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1016 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1017 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1018 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1019 // Block: (32, 4, 1)
1020 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1021 // source: LeetCode/kernels/sgemv/sgemv.cu
1022 template <const int ROW_PER_WARP = 2>
1023 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1024     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1025     int tx = threadIdx.x;
1026     int ty = threadIdx.y;
1027     int lane = tx % WARP_SIZE;
1028     int k = lane % K_WARP_SIZE; // 0~15
1029     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1030     if (m < M) {
1031         float sum = A[m * K + k] * x[k];
1032         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1033         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1034         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1035         if (k == 0)
1036             y[m] = sum;
1037     }
1038 }
1039
1040 // =====
1041 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1042 // =====
1043 // 面试要点 (GEMM 优化五层金字塔):
1044 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1045 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1046 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1047 //   load/store 指令数 Level 4 — Tensor Core (MMA
1048 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1049 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1050 //
1051 // 计算密度递进:
1052 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1053 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1054 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1055 //
1056 // =====
1057 // Phase 7a: SGEMM (非 Tensor Core 路径)
1058 // =====
1059
1060 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1061 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1062 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1063 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1064 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1065 // Block: (32, 32, 1), 1024 线程
1066 // source: LeetCode/kernels/sgemm/sgemm.cu
1067 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1068     constexpr int BM = 32; // vec 版: 32×4 = 128
1069     constexpr int BN = 32; // vec 版: 32×4 = 128
1070     constexpr int BK = 32;

```

```

1071 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1072
1073 int bx = blockIdx.x;
1074 int by = blockIdx.y;
1075 int tx = threadIdx.x;
1076 int tid = threadIdx.y * blockDim.x + tx;
1077
1078 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1079 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1080 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1081 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1082 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1083 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1084 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1085
1086 float sum = 0.f; // 遍历完整的K, slice K;
1087 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1088     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1089     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1090     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1091     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1092     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1093     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1094     __syncthreads(); // 确保整个 smem tile 加载完毕
1095
1096 #pragma unroll
1097     for (int k = 0; k < BK; ++k) {
1098         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1099         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1100         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1101     }
1102     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1103 }
1104 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1105 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1107 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1108 }
1109
1110 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1111 // 在 Level 1 基础上引入两层优化:
1112 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1113 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1114 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1115 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1116 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1117 // 1024 x 16 = 16384 = 128 x 128 ✓
1118 //
1119 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1120 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1121 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1122 //
1123 // 加载映射 (每线程 4 个元素, float4):
1124 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1125 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1126 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1127 //
1128 // △ Bank Conflict 提示 (面试加分点):
1129 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1130 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1131 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1132 //
1133 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)

```

```

1134 // Block: (32, 32, 1), 1024 线程
1135 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1136 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1137 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1138     constexpr int BM = 128;
1139     constexpr int BN = 128;
1140     constexpr int BK = 32;
1141     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1142     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1143
1144     int bx = blockIdx.x;
1145     int by = blockIdx.y;
1146     int tx = threadIdx.x;
1147     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1148
1149     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1150     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1151     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1152     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1153     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1154     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1155
1156     int load_gmem_a_m = by * BM + load_smem_a_m;
1157     int load_gmem_b_n = bx * BN + load_smem_b_n;
1158
1159     // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1160     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1161     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1162
1163     float sum[4][4] = {0.f};
1164     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1165         int load_gmem_a_k = bk * BK + load_smem_a_k;
1166         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1167         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1168         int load_gmem_b_k = bk * BK + load_smem_b_k;
1169         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1170         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1171         __syncthreads();
1172
1173     #pragma unroll
1174         for (int k = 0; k < BK; ++k) {
1175             // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1176             float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1177                                s_a[comp_smem_a_m_base + 1][k],
1178                                s_a[comp_smem_a_m_base + 2][k],
1179                                s_a[comp_smem_a_m_base + 3][k]};
1180             float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1181                                s_b[k][comp_smem_b_n_base + 1],
1182                                s_b[k][comp_smem_b_n_base + 2],
1183                                s_b[k][comp_smem_b_n_base + 3]};
1184
1185             #pragma unroll
1186             for (int i = 0; i < 4; ++i) {
1187                 #pragma unroll
1188                 for (int j = 0; j < 4; ++j) {
1189                     sum[i][j] += a_vals[i] * b_vals[j];
1190                 }
1191             }
1192         }
1193         __syncthreads();
1194     }
1195
1196     // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1197     int store_gmem_c_m = by * BM + comp_smem_a_m_base;

```

```

1197     int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1198 #pragma unroll
1199     for (int i = 0; i < 4; ++i) {
1200         int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1201         float4 reg_c;
1202         reg_c.x = sum[i][0];
1203         reg_c.y = sum[i][1];
1204         reg_c.z = sum[i][2];
1205         reg_c.w = sum[i][3];
1206         FLOAT4(c[store_gmem_c_addr]) = reg_c;
1207     }
1208 }
1209
1210 // =====
1211 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1212 // =====
1213 // 面试要点:
1214 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1215 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
1216 //   - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
1217 //     寄存器)
1218 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1219 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1220 //
1221 // ★ TN 布局详解 (面试高频考点):
1222 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1223 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1224 //   N = Normal (列优先, BLAS 原生格式)
1225 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1226 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1227 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1228 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1229 //   视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1230 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1231 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1232 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1233 //
1234 //   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1235 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1236 //   N = col-major (列优先, BLAS native = normal)
1237 //
1238 //   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1239 //   - A: row-major [M, K], B: row-major [K, N]
1240 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1241 //   - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1242 //
1243 //   TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1244 //   - A [M×K]: row-major → 全局索引 A[m*K+k], smem s_a[BM][BK]
1245 //   - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (Δ 内维连续的是 K)
1246 //   - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1247 //   MMA 指令: mma.sync.aligned.m16n8k16.row.col
1248 //   - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1249 //
1250 //   WGMMMA (Warp Group MMA, Hopper+):
1251 //   - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1252 //   - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1253 //   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1254 //     threads) 做计算
1255 //   - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1256 //
1257 // =====
1258 // Phase 7b-1: MMA PTX 宏定义
1259 // =====

```

```

1260
1261 // ---- gmem → smem: cp.async ----
1262 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1263 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1264 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1265 //     N=0 → 等所有 group 完成。与 wmma.wait_group 语义一致。
1266 //   - wait_all: 等价于 commit_group + wait_group 0。
1267 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1268 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1269 #define CP_ASYNC_WAIT_GROUP(n) \
1270     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1271
1272 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1273 #define CP_ASYNC_CG(dst, src, bytes) \
1274     asm volatile( \
1275         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1276         "l"(src), "n"(bytes))
1277
1278 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1279 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1280 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1281 // aligned: 要求 128-bit 对齐, trans: 转置加载
1282 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1283     asm volatile( \
1284         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1285         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1286         : "r"(addr))
1287
1288 #define LDMATRIX_X2(R0, R1, addr) \
1289     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1290         : "=r"(R0), "=r"(R1) \
1291         : "r"(addr))
1292
1293 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1294 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1295 #define LDMATRIX_X2_T(R0, R1, addr) \
1296     asm volatile( \
1297         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1298         : "=r"(R0), "=r"(R1) \
1299         : "r"(addr))
1300
1301 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1302 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1303 // row.col: A 是 row-major, B 是 col-major
1304 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1305 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1306 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1307 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1308     asm volatile( \
1309         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1310         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1311         : "=r"(RD0), "=r"(RD1) \
1312         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1313         "r"(RC1))
1314
1315 // ---- MMA 辅助函数 ----
1316 // div_ceil: 整数除法向上取整
1317 #define HOST_DEVICE_INLINE __device__ __host__ inline
1318 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1319     return (a % b != 0) ? (a / b + 1) : (a / b);
1320 }
1321
1322 // =====

```



```

1323 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1324 // =====
1325 // 面试重点 — Tile Hierarchy:
1326 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1327 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1328 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1329 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1330 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1331 //
1332 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1333 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1334 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1335 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1336 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1337 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1338 //
1339 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1340 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1341 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1342 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1343 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1344 //
1345 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1346 // Block: (256, 1, 1), 8 warps
1347 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1348 // =====
1349 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1350           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1351           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1352           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1353 __global__ void __launch_bounds__(256)
1354   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1355   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1356   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1357   const int by = blockIdx.y;
1358   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1359   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1360   constexpr int BK = MMA_K; // 16
1361
1362   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1363   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1364   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1365   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1366   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1367   extern __shared__ half smem[];
1368   half *s_a = smem;
1369   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1370   constexpr int s_a_stage_offset = BM * BK; // 128*16
1371   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1372   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1373   const int warp_id = tid / WARP_SIZE; // 0~7
1374   const int lane_id = tid % WARP_SIZE; // 0~31
1375   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1376   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1377
1378   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1379   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1380   int load_smem_a_m = tid / 2; // 0~127
1381   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1382   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1383   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1384   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1385   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移

```

```

1386 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1387     return;
1388
1389 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1390 // CUDA中每个block能放的线程数是有上限的(warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1391 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1392 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1393 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1394
1395 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1396 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1397 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1398
1399 // 预加载前 (K_STAGE-1) 个 stage
1400 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1401 #pragma unroll
1402 for (int k = 0; k < (K_STAGE - 1); ++k) {
1403     int load_gmem_a_k = k * BK + load_smem_a_k;
1404     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1405     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1406         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1407     );
1408     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1409
1410     int load_gmem_b_k = k * BK + load_smem_b_k;
1411     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1412     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1413     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1414         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1415     );
1416     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1417
1418     CP_ASYNC_COMMIT_GROUP();
1419 }
1420
1421 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1422 __syncthreads();
1423
1424 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1425 const int NUM_K_TILES = div_ceil(K, BK);
1426 #pragma unroll
1427 for (int k = 0; k < NUM_K_TILES; ++k) {
1428     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1429     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1430
1431     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1432     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1433     if (k + K_STAGE - 1 < NUM_K_TILES) {
1434         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1435         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1436         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1437         // B^T: row-major [n][k], 内维连续的是 K △
1438         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1439
1440         uint32_t load_smem_a_ptr =
1441             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1442                 load_smem_a_m * BK + load_smem_a_k) *
1443                 sizeof(half));
1444         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1445
1446         uint32_t load_smem_b_ptr =
1447             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1448                 load_smem_b_n * BK + load_smem_b_k) *

```



```

1449         sizeof(half));
1450     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1451     CP_ASYNC.COMMIT_GROUP();
1452 }
1453
1454 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1455 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1456 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1457 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1458 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1459
1460 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1461 #pragma unroll
1462 for (int i = 0; i < VAL_TILE_M; ++i) {
1463     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1464     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1465     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1466     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1467     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1468     uint32_t lane_smem_a_ptr =
1469         (smem_a_base_ptr +
1470          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1471           sizeof(half));
1472     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1473 }
1474
1475 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1476 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1477 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1478 #pragma unroll
1479 for (int j = 0; j < VAL_TILE_N; ++j) {
1480     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1481     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1482     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1483     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1484     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1485     uint32_t lane_smem_b_ptr =
1486         (smem_b_base_ptr +
1487          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1488           sizeof(half));
1489     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1490 }
1491
1492 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1493 #pragma unroll
1494 for (int i = 0; i < VAL_TILE_M; ++i) {
1495     #pragma unroll
1496     for (int j = 0; j < VAL_TILE_N; ++j) {
1497         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1498                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1499                 RB[j][0], RB[j][1], // B fragment
1500                 RC[i][j][0], RC[i][j][1]);
1501     }
1502 }
1503
1504 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1505 if (k + K_STAGE - 1 < NUM_K_TILES) {
1506     CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1507 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1508     CP_ASYNC.WAIT_GROUP(0);
1509 }
1510 __syncthreads();
1511 }

```

```

1512 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1513 {
1514     for (int i = 0; i < VAL_TILE_M; ++i) {
1515         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1516         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1517         // =====
1518         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1519         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1520         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1521         //
1522         //      cols: c0-1    c2-3    c4-5    c6-7
1523         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1524         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1525         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1526         //  ...                               |
1527         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1528         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1529         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1530         //  r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1531         //  ...                               |
1532         //  r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1533         //
1534         // Within a 4-lane group (e.g. T0-T3 for row 0):
1535         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1536         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1537         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1538         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1539         // =====
1540     }
1541     #pragma unroll
1542     for (int j = 0; j < VAL_TILE_N; ++j) {
1543         RC0[j][0] = RC[i][j][0];
1544         RC1[j][0] = RC[i][j][1];
1545         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1546         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1547         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1548         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1549         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1550         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1551     }
1552     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1553     // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1554     //
1555     // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1556     // |-----|-----|-----|-----|
1557     // | 0~3   | 0       | row 0     | row 8     |
1558     // | 4~7   | 1       | row 1     | row 9     |
1559     // | 8~11  | 2       | row 2     | row 10    |
1560     // | 12~15 | 3       | row 3     | row 11    |
1561     // | 16~19 | 4       | row 4     | row 12    |
1562     // | 20~23 | 5       | row 5     | row 13    |
1563     // | 24~27 | 6       | row 6     | row 14    |
1564     // | 28~31 | 7       | row 7     | row 15    |
1565     //
1566     if (lane_id % 4 == 0) {
1567         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1568         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1569         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1570     #pragma unroll
1571     for (int j = 0; j < VAL_TILE_N; ++j) {
1572         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1573         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1574         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;

```

```

1575     int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1576     int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1577     // 128-bit store: 一次写入 8 个 half
1578     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1579         *reinterpret_cast<float4*>(&RC0[j][0]);
1580     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1581         *reinterpret_cast<float4*>(&RC1[j][0]);
1582 }
1583 }
1584 }
1585 }
1586 }
1587
1588 // =====
1589 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1590 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1591 // =====
1592 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1593 //   - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1594 //     不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1595 //   - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1596 //     将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1597 //       rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1598 //       rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1599 //       rows 8~11: repeat rows 0~3 pattern
1600 //       rows 12~15: repeat rows 4~7 pattern
1601 //   - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1602 //   - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1603 //   - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1604 //     对 MMA m16n8k16 的 BK=16 而言刚好满足。
1605 //
1606 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1607 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1608
1609 // ---- Swizzle 辅助函数 ----
1610
1611 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1612 // i: row index; j: col index.
1613 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1614 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1615 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1616 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1617 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1618 template <const int kColStride = 16, const int kStep = 8>
1619 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1620     // for col_stride > 16, we have to permute it using col major ZigZag order.
1621     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1622     static_assert(kColStride <= 16, "kColStride must <= 16");
1623     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1624     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1625     static_assert(kColStride % kStep == 0,
1626         "kColStride must be multiple of kStep.");
1627     if constexpr (kStep == 8) {
1628         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1629     } else {
1630         static_assert(kStep == 4);
1631         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1632     }
1633 }
1634
1635 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8) 。
1636 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1637 // -----

```

```

1638 // -col 0~16, step 8--
1639 // -----
1640 // | row 0 | (0, 8) |
1641 // | row 1 | (0, 8) |
1642 // | row 2 | (0, 8) |
1643 // | row 3 | (0, 8) |
1644 // -----
1645 // | row 4 | (8, 0) |
1646 // | row 5 | (8, 0) |
1647 // | row 6 | (8, 0) |
1648 // | row 7 | (8, 0) |
1649 // -----
1650 // | row 8 | (0, 8) |
1651 // | row 9 | (0, 8) |
1652 // | row 10 | (0, 8) |
1653 // | row 11 | (0, 8) |
1654 // -----
1655 // | row 12 | (8, 0) |
1656 // | row 13 | (8, 0) |
1657 // | row 14 | (8, 0) |
1658 // | row 15 | (8, 0) |
1659 // -----
1660 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1661 template <const int kMmaAtomK = 16>
1662 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1663     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1664 }
1665
1666 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern) 。
1667 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1668 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8) 。
1669 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1670 template <const int kMmaAtomK = 16>
1671 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1672     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1673 }
1674
1675 // =====
1676 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1677 // =====
1678 // 与 Phase 7b-2 (hgemm_mma_stages_tn) 的核心区别:
1679 // - 所有 smem 地址计算加 swizzle_permuted_A_j/swizzle_permuted_B_j, 消除 bank conflict
1680 // - VAL_TILE_M/N → VAL_TILE_M/N (与源 kernel 命名一致)
1681 // - 其余 tile hierarchy、pipeline、epilogue 与统一循环版完全一致
1682 //
1683 // Tile Hierarchy:
1684 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1685 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1686 //   WARP Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1687 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1688 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1689 //
1690 // TN 布局: C[M×N] = A[M×K] × B^T[N×K]
1691 // - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1692 // - B^T[N][K]: row-major → 全局索引 B[n*K+k]
1693 // - ldmatrix A: x4 (非转置), A row-major 原生匹配
1694 // - ldmatrix B: x2 (非转置), B^T row-major 逐行加载 = B 的列
1695 // - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1696 //
1697 // 统一循环设计 (同 hgemm_mma_stages_tn) :
1698 // - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1699 // - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1700 // - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载

```

```

1701 // - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1702 //
1703 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1704 // Block: (256, 1, 1), 8 warps
1705 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1706 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1707           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1708           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1709           const int K_STAGE = 3,
1710           const bool BLOCK_SWIZZLE = false>
1711 __global__ void __launch_bounds__(256)
1712   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1713   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1714   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1715   const int by = blockIdx.y;
1716   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1717   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1718   constexpr int BK = MMA_K; // 16
1719
1720   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B (与 hgemm_mma_stages_tn 相同)
1721   extern __shared__ half smem[];
1722   half *s_a = smem;
1723   half *s_b = smem + K_STAGE * BM * BK;
1724   constexpr int s_a_stage_offset = BM * BK; // 128*16
1725   constexpr int s_b_stage_offset = BN * BK; // 128*16 Δ B^T row-major
1726   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1727   const int warp_id = tid / WARP_SIZE; // 0~7
1728   const int lane_id = tid % WARP_SIZE; // 0~31
1729   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1730   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1731
1732   // 线程到 global memory 的映射 (用于加载 A 和 B)
1733   int load_smem_a_m = tid / 2; // 0~127
1734   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1735   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1736   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1737   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号
1738   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号
1739   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1740     return;
1741
1742   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1743   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1744   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1745   // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1746   uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1747
1748   // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1749   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1750   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1751
1752   // 预加载前 (K_STAGE-1) 个 stage
1753   // ★ swizzle 区别: cp.async 的目标 smem 地址加 swizzle_permuted_A_j/swizzle_permuted_B_j
1754   #pragma unroll
1755   for (int k = 0; k < (K_STAGE - 1); ++k) {
1756     int load_gmem_a_k = k * BK + load_smem_a_k;
1757     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1758     int load_gmem_b_k = k * BK + load_smem_b_k;
1759     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1760
1761     uint32_t load_smem_a_ptr =
1762       (smem_a_base_ptr +
1763        (k * s_a_stage_offset + load_smem_a_m * BK +

```

```

1764         swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1765         sizeof(half));
1766     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1767
1768     uint32_t load_smem_b_ptr =
1769         (smem_b_base_ptr +
1770         (k * s_b_stage_offset + load_smem_b_n * BK +
1771         swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1772         sizeof(half));
1773     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1774
1775     CP_ASYNC.COMMIT_GROUP();
1776 }
1777
1778 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1779 __syncthreads();
1780
1781 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1782 const int NUM_K_TILES = div_ceil(K, BK);
1783 #pragma unroll
1784 for (int k = 0; k < NUM_K_TILES; ++k) {
1785     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1786     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1787
1788     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1789     // swizzle 区别: smem 地址加 swizzle_permuted*_j
1790     if (k + K_STAGE - 1 < NUM_K_TILES) {
1791         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1792         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1793         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1794         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1795
1796         uint32_t load_smem_a_ptr =
1797             (smem_a_base_ptr +
1798             (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1799             swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1800             sizeof(half));
1801         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1802
1803         uint32_t load_smem_b_ptr =
1804             (smem_b_base_ptr +
1805             (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1806             swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1807             sizeof(half));
1808         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1809         CP_ASYNC.COMMIT_GROUP();
1810     }
1811
1812     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1813     // * swizzle 区别: ldmatrix 的源 smem 地址也加 swizzle_permuted*_j
1814     uint32_t RA[VAL_TILE_M][4];
1815     uint32_t RB[VAL_TILE_N][2];
1816
1817     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1818     #pragma unroll
1819     for (int i = 0; i < VAL_TILE_M; ++i) {
1820         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1821         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1822         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1823         uint32_t lane_smem_a_ptr =
1824             (smem_a_base_ptr +
1825             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1826             swizzle_permuted_A_j<MMA_K>(lane_smem_a_m, lane_smem_a_k)) *

```

```

1827         sizeof(half));
1828     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1829 }
1830
1831 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1832 #pragma unroll
1833 for (int j = 0; j < VAL_TILE_N; ++j) {
1834     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1835     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1836     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1837     uint32_t lane_smem_b_ptr =
1838         (smem_b_base_ptr +
1839          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1840           swizzle_permuted_B_j < MMA_K > (lane_smem_b_n, lane_smem_b_k)) *
1841           sizeof(half));
1842     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1843 }
1844
1845 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1846 #pragma unroll
1847 for (int i = 0; i < VAL_TILE_M; ++i) {
1848     #pragma unroll
1849     for (int j = 0; j < VAL_TILE_N; ++j) {
1850         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1851                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1852                  RB[j][0], RB[j][1], // B fragment
1853                  RC[i][j][0], RC[i][j][1]);
1854     }
1855 }
1856
1857 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1858 if (k + K_STAGE - 1 < NUM_K_TILES) {
1859     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1860 } else {
1861     CP_ASYNC_WAIT_GROUP(0);
1862 }
1863 __syncthreads();
1864 }
1865
1866 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1867 // 与 hgemm_mma_stages_tn 完全一致的 epilogue 模式
1868 {
1869     for (int i = 0; i < VAL_TILE_M; ++i) {
1870         uint32_t RC0[VAL_TILE_N][4];
1871         uint32_t RC1[VAL_TILE_N][4];
1872     #pragma unroll
1873     for (int j = 0; j < VAL_TILE_N; ++j) {
1874         RC0[j][0] = RC[i][j][0];
1875         RC1[j][0] = RC[i][j][1];
1876         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1877         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1878         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1879         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1880         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1881         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1882     }
1883     if (lane_id % 4 == 0) {
1884         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1885         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1886     #pragma unroll
1887     for (int j = 0; j < VAL_TILE_N; ++j) {
1888         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1889         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;

```



```

1890     int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1891     int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1892     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1893         *reinterpret_cast<float4*>(&RC0[j][0]);
1894     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1895         *reinterpret_cast<float4*>(&RC1[j][0]);
1896 }
1897 }
1898 }
1899 }
1900 }
1901
1902 // =====
1903 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
1904 // =====
1905 // 面试要点 (CuTe vs 手写 MMA PTX) :
1906 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
1907 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
1908 //   - 与手写 MMA PTX (Phase 7b) 的对比:
1909 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
1910 //     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
1911 //       cute::copy / cute::gemm 自动展开为高效指令序列
1912 //   - 核心抽象 ("CuTe 五要素") :
1913 //     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
1914 //     2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
1915 //     3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
1916 //     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
1917 //     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
1918 //
1919 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
1920 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
1921 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
1922 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
1923 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
1924 //
1925 // 调参口诀 (面试速记) :
1926 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
1927 //   Swizzle<B,M,S>: B=bank 维度 bit, M=M维 bit, S=stride bit → Swizzle<3,3,3> = 512 values = 1024B
1928 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
1929 //   EURepeat: 增加 warp 数 → 更多并行但每个 warp 做更少 MMA → 寄存器压力降低
1930 //
1931 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
1932 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
1933 //   - CuTe Swizzle: SmemLayout 声明式指定 swizzle pattern, cute::copy 自动应用
1934 //   - CuTe 优势: 类型安全, 编译器可做更激进的优化, 布局变更只需改类型定义
1935 //
1936 // ★ 本实现的 Tile 配置:
1937 //   BM=128, BN=256, BK=32, kStage=2
1938 //   MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1, ValTile=32x32x16
1939 //   Smem Swizzle<3,3,3>: 512-value XOR permutation, 消除 ldmatrix bank conflict
1940 //   Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)
1941 //   Smem: ~49KB (Stage=2, A[128×32] + B[256×32] × half)
1942 //
1943 // 参考: LeetCode/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
1944 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
1945 // =====
1946
1947 #if defined(NOTES_V2_ENABLE_CUTE)
1948
1949 #include <cute/tensor.hpp>
1950
1951 // =====
1952 // Phase 7d-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline

```

```

1953 // =====
1954 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
1955 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价 B col-major  $[K \times N]$ )
1956 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
1957 //
1958 // Pipeline 流程 (每个 stage 对应一次完整的 G→S→R→MMA→R→S→G 链条):
1959 //   1. PREFETCH: 预加载 kStage-1 个 tile (G→S via cp.async)
1960 //   2. 主循环 over K tiles:
1961 //       a. S→R: cute::copy(s2r_tiled_copy, sA, rA) ← 自动展开为 ldmatrix
1962 //       b. MMA: cute::gemm(tiled_mma, rD, rA, rB, rD) ← 自动展开为 mma.sync
1963 //       c. G→S (next tile): cute::copy(g2s_tiled_copy, gA, sA) ← cp.async
1964 //       d. Pipeline wait + smem stage advance
1965 //   3. Epilogue: R→S (via UniversalCopy) → S→G (128-bit store)
1966 //
1967 // 面试对比 — 手写 MMA vs CuTe 代码量:
1968 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
1969 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
1970 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
1971           typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
1972           typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
1973           typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
1974           typename S2GCopyC>
1975 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
1976                                           int n, int k) {
1977     using namespace cute;
1978
1979     // 动态 shared memory: A tile + B tile (C epilogue 复用 A tile 的空间)
1980     extern __shared__ T shm_data[];
1981     T *Ashm = shm_data;
1982     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
1983
1984     int idx = threadIdx.x;
1985     int ix = blockIdx.x; // N 方向 block 索引 (本实现默认不开启 BlockSwizzle)
1986     int iy = blockIdx.y; // M 方向 block 索引
1987     if (iy * BM >= m || ix * BN >= n) return;
1988
1989     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
1990     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
1991     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
1992                             make_stride(k, Int<1>{}));
1993     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
1994                             make_stride(k, Int<1>{}));
1995     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
1996                             make_stride(n, Int<1>{}));
1997
1998     // local_tile: 从全局 Tensor 切出当前 CTA 负责的 tile
1999     // make_coord(iy, _): M 维固定为 iy, K 维自动按 BK 分 tile (_ = 通配符)
2000     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2001     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2002     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2003
2004     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2005     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2006     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2007
2008     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2009     TiledMMA tiled_mma;
2010     auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2011     auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2012     auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2013     auto tCrD = thr_mma.partition_fragment_C(gD);           // (MMA, MMA_M, MMA_N)
2014     cclear(tCrD); // 累加器清零
2015

```

```

2016 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (使用 cp.async 128-bit)
2017 G2SCopyA g2s_tiled_copy_a;
2018 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2019 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, k_tile)
2020 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2021
2022 G2SCopyB g2s_tiled_copy_b;
2023 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2024 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB);
2025 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2026
2027 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2028 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2029 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2030 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2031 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2032 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2033
2034 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2035 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2036 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2037 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2038
2039 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满流水线
2040 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2041 #pragma unroll
2042 for (int istage = 0; istage < kStage - 1; ++istage) {
2043     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2044               tAsA_copy(_, _, _, istage));
2045     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2046               tBsB_copy(_, _, _, istage));
2047     cp_async_fence();
2048     ++itile_to_read;
2049     ++ismem_write;
2050 }
2051 cp_async_wait<kStage - 2>();
2052 __syncthreads();
2053
2054 // 主循环: over K tiles, 每个 K tile 内再分 MMA_K 步迭代
2055 int ntile = k / BK; // K 方向总 tile 数
2056 int ik = 0;
2057 // 首轮 S→R 加载 (在进入循环前先加载第一个 ik slice)
2058 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik, ismem_read), tCrA_view(_, _, ik));
2059 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik, ismem_read), tCrB_view(_, _, ik));
2060
2061 #pragma unroll 1
2062 for (int itile = 0; itile < ntile; ++itile) {
2063     int nk = size<2>(tCrA); // 每个 K tile 内的 MMA_K 步数
2064
2065     #pragma unroll
2066     for (int ik = 0; ik < nk; ++ik) {
2067         int ik_next = (ik + 1) % nk;
2068
2069         if (ik == nk - 1) {
2070             // 等待下一批 smem 数据就绪 (pipeline 同步)
2071             cp_async_wait<kStage - 2>();
2072             __syncthreads();
2073             ismem_read = (ismem_read + 1) % kStage;
2074         }
2075
2076         // S→R: 加载下一组 A/B fragment 到寄存器 (用 ik_next 预取)
2077         cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik_next, ismem_read),
2078                   tCrA_view(_, _, ik_next));

```

```

2079     cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik_next, ismem_read),
2080               tCrB_view(_, _, ik_next));
2081
2082     if (ik == 0) {
2083         // G→S: 提交下一个 K tile 的异步拷贝 (流水线加载)
2084         if (itile_to_read < ntile) {
2085             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2086                       tAsA_copy(_, _, _, ismem_write));
2087             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2088                       tBsB_copy(_, _, _, ismem_write));
2089             ++itile_to_read;
2090             ismem_write = (ismem_write + 1) % kStage;
2091         }
2092         cp_async_fence();
2093     }
2094
2095     // MMA: 发射 Tensor Core 指令 (自动展开为 mma.sync.aligned.m16n8k16)
2096     cute::gemm(tiled_mma, tCrD, tCrA(_, _, ik), tCrB(_, _, ik), tCrD);
2097 }
2098 }
2099
2100 // Epilogue: D寄存器 → shared memory → global memory
2101 // 使用 A tile 的最后一个 stage 作为 C 的 scratchpad (节省 smem)
2102 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2103
2104 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2105 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2106 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2107 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2108 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2109
2110 // S2G TiledCopy: shared memory → global memory (128-bit store)
2111 S2GCopyC s2g_tiled_copy_c;
2112 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_thread_slice(idx);
2113 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2114 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2115
2116 // group_modes: 将多维 Tensor 的某些 mode 合并, 简化遍历
2117 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2118 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2119
2120 int step = size<3>(tCsC_r2s); // pipeline depth of C smem
2121 #pragma unroll
2122 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2123     // R→S: 寄存器写回 shared memory
2124     #pragma unroll
2125     for (int j = 0; j < step; ++j) {
2126         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2127         cute::copy(tCrC_r2sx(_, i + j), t);
2128         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2129     }
2130     __syncthreads();
2131
2132     // S→G: shared memory 写回 global memory
2133     #pragma unroll
2134     for (int j = 0; j < step; ++j) {
2135         cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2136     }
2137     __syncthreads();
2138 }
2139 }
2140
2141 // =====

```

```

2142 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2143 // =====
2144 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2145 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2146 // kernel 接收这些类型的实例 (通常为空 struct), 在编译期完成全部映射推导。
2147 //
2148 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2149 // 答: 手写需要:
2150 //   1) 手动计算每线程的 smem 地址偏移
2151 //   2) 手动计算 ldmatrix 的 lane 映射
2152 //   3) 手动插入 swizzle XOR 到每个地址计算点
2153 //   4) 手动做 epilogue 的 warp shuffle 编排
2154 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2155 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2156 //
2157 // 模板参数推导链 (面试速记):
2158 //   SM80_16x8x16_F16F16F16F16_TN ← MMA 指令 atom
2159 //   → MMA_Atom<MMA_Traits<mma_op>>
2160 //   → make_tiled_mma(mma_atom, EURepeat{2,2,1}, ValTile{32,32,16})
2161 //   → TiledMMA (128 threads, 4 warps × 2×2 slices)
2162 //
2163 //   SM80_CP_ASYNC_CACHEGLOBAL<uint128_t> ← G→S copy atom
2164 //   → Copy_Atom<Copy_Traits<g2s_copy_op>, T>
2165 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2166 //   → G2SCopy: 32×4=128 threads, each copies 1×8=8 halves = 128 bits
2167 //
2168 //   Swizzle<3,3,3> + Atom{8,BK} → composition → tile_to_shape{BM,BK,kStage}
2169 //   → SmemLayoutA/B: 带 XOR swizzle 的 smem 排布
2170 //
2171 //   SM75_U32x4_LDSM_N ← S→R copy atom (ldmatrix 的 CuTe 封装)
2172 //   → Copy_Atom<Copy_Traits<s2r_copy_op>, T>
2173 //
2174 //   UniversalCopy<uint128_t> ← S→G copy atom (128-bit store)
2175 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2176 //
2177 // ★ Tile 尺寸速查:
2178 //   BM=128: M 方向 block tile (16 × 2 EU × 4 Val = 128)
2179 //   BN=256: N 方向 block tile (8 × 2 EU × 16 Val... etc. 实际 8 × 2 × 16 = 256)
2180 //   BK=32: K 方向 block tile (= kMmaPK = 1 × 1 × 16 = 16? 不对, BK=32 由 SmemLayout 控制)
2181 //   kStage=2: 双缓冲 pipeline
2182 // =====
2183 template <typename T, const int Stages = 2>
2184 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2185     using namespace cute;
2186
2187     auto BM = Int<128>{};
2188     auto BN = Int<256>{};
2189     auto BK = Int<32>{};
2190     auto KStage = Int<Stages>{};
2191     auto kSmemLayoutCBatch = Int<4>{}; // C smem pipeline depth
2192
2193     // SmemLayout: Swizzle<3,3,3> 做 512-value (1024-byte) XOR 置换消除 bank conflict
2194     // composition: 将 swizzle pattern 应用到 8×BK 的 base layout
2195     // tile_to_shape: 将 atom layout 扩展到完整的 BM×BK×kStage
2196     using SmemLayoutAtom = decltype(composition(
2197         Swizzle<3, 3, 3>{},
2198         make_layout(make_shape(Int<8>{}, Int<BK>{}),
2199             make_stride(Int<BK>{}, Int<1>{}))));
2200     using SmemLayoutA = decltype(tile_to_shape(
2201         SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2202     using SmemLayoutB = decltype(tile_to_shape(
2203         SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2204

```

```

2205 // MMA: SM80_16x8x16_F16F16F16F16_TN
2206 // TN = A row-major, B col-major (与 Phase 7b 手写 MMA 完全相同的布局约定)
2207 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2208 using mma_traits = MMA_Traits<mma_op>;
2209 using mma_atom = MMA_Atom<mma_traits>;
2210 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2211
2212 static constexpr int kMmaEURepeatM = 2;
2213 static constexpr int kMmaEURepeatN = 2;
2214 static constexpr int kMmaEURepeatK = 1;
2215 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2216 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2217 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2218
2219 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2220     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2221 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2222 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2223
2224 // G2S Copy: cp.async 128-bit, 32x4 threads each loading 1x8 halves
2225 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2226 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2227 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2228 using G2SCopyA = decltype(make_tiled_copy(
2229     g2s_copy_atom{},
2230     make_layout(make_shape(Int<32>{}, Int<4>{}),
2231         make_stride(Int<4>{}, Int<1>{})),
2232     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2233 using G2SCopyB = G2SCopyA;
2234
2235 // S2R Copy: ldmatrix (SM75_U32x4_LDSTM_N), 由 make_tiled_copy_A/B 自动与 TiledMMA 对齐
2236 using s2r_copy_op = SM75_U32x4_LDSTM_N;
2237 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2238 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2239 using S2RCopyAtomA = s2r_copy_atom;
2240 using S2RCopyAtomB = s2r_copy_atom;
2241
2242 // Epilogue smem layout for C: Swizzle<3,3,3> on 32x32 atom, 4 pipeline stages
2243 using SmemLayoutAtomC = decltype(composition(
2244     Swizzle<3, 3, 3>{},
2245     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2246         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2247 using SmemLayoutC = decltype(tile_to_shape(
2248     SmemLayoutAtomC{},
2249     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2250
2251 // R2S Copy: 寄存器 → shared memory (epilogue 阶段, UniversalCopy<int> 做逐 int 拷贝)
2252 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2253
2254 // S2G Copy: shared memory → global (128-bit store)
2255 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2256 using S2GCopyC = decltype(make_tiled_copy(
2257     S2GCopyAtomC{},
2258     make_layout(make_shape(Int<32>{}, Int<4>{}),
2259         make_stride(Int<4>{}, Int<1>{})),
2260     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2261
2262 // Grid/Block 配置
2263 int BX = (N + BN - 1) / BN;
2264 int BY = (M + BM - 1) / BM;
2265 dim3 block(size(MMA{})); // 128 threads
2266 dim3 grid(BX, BY);
2267

```



```

2268 // Smem 大小: A tile + B tile (C epilogue 复用 A tile 空间)
2269 static constexpr int shm_size_AB =
2270     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2271 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2272 // C smem ≤ A 的一个 pipe, 可以安全复用
2273 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2274     "C shared memory must fit within one A pipe");
2275 static constexpr int kShmSize =
2276     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2277
2278 cudaFuncSetAttribute(
2279     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2280         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2281         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2282         S2GCopyAtomC, S2GCopyC>,
2283     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2284
2285 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2286     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2287     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC>
2288     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2289 }
2290
2291 #endif /* NOTES_V2_ENABLE_CUTE */
2292
2293 // =====
2294 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2295 // =====
2296 // 面试要点 (WGMMMA vs MMA 对比) :
2297 //   - MMA: warp 级 (32 threads), 同步执行
2298 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2299 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2300 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2301 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2302 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2303
2304 // ---- WGMMMA 辅助函数 ----
2305 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2306 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2307 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2308 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2309 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2310 //     都是 "wait until only N or fewer groups are pending"。
2311 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2312 #define WGMMMA_COMMIT_GROUP() \
2313     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2314 #define WGMMMA_WAIT_GROUP(n) \
2315     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2316
2317 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2318 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2319 //   encoded = (x & 0x3FFFF) >> 4
2320 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2321 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2322 // 可省略以扩大可表示范围。
2323 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2324 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2325
2326 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2327 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2328 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2329 //
2330 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :

```



```

2331 // -----
2332 // | 位域          | 范围      | 大小 | 含义
2333 // |-----|-----|-----|-----
2334 // | start_address  | [0, 14)   | 14   | smem 基址编码
2335 // | (unused)       | [14, 16)  | 2    | -
2336 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2337 // | (unused)       | [30, 32)  | 2    | -
2338 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2339 // | (unused)       | [46, 49)  | 3    | -
2340 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2341 // | (unused)       | [52, 62)  | 10   | -
2342 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2343 // -----
2344 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2345 //
2346 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2347 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2348 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2349 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2350 // - 这是 stride_byte_offset=1024 的来源
2351 //
2352 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2353 //   此处填 16 仅为占位, 编码后为 1。
2354 //
2355 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2356 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2357 //
2358 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2359 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2360 __device__ inline uint64_t make_smem_desc(half *ptr) {
2361     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2362     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2363     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2364
2365     // 从全零开始: 所有位域初始化为 0。
2366     uint64_t desc = 0x0000000000000000;
2367
2368     // — bits [0, 14): start_address —
2369     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2370     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2371     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2372     desc |= SMEM_DESC_ENCODE(addr);
2373
2374     // — bits [16, 30): leading_byte_offset —
2375     // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2376     // << 16 将编码值放到 bits [16, 30)。
2377     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2378     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2379     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2380     //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2381     //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2382     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2383
2384     // — bits [32, 46): stride_byte_offset —
2385     // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2386     // << 32 将编码值放到 bits [32, 46)。
2387     // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2388     // 1024 的来源 (K-Major + 128B swizzle + half) :
2389     //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2390     //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2391     // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2392     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2393

```

```

2394 // — bits [62, 64): layout_type (swizzle mode) —
2395 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2396 // 1 = 128B swizzle mode。
2397 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2398 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2399 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2400 desc |= 1llu << 62;
2401
2402 return desc;
2403 }
2404
2405 // ---- WGMMA PTX 指令宏 ----
2406 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2407 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2408 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2409 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2410 // 每线程 64 half = 32 uint32。
2411 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2412 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2413 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2414 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2415 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2416 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2417                                     TransB) \
2418 { \
2419     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2420     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2421     asm volatile( \
2422         "{\n" \
2423         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2424         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2425         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2426         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2427         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2428         "%32, " \
2429         "%33, " \
2430         "%34, %35, %36, %37, %38;\n" \
2431         "}\n" \
2432         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2433         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2434         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2435         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2436         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2437         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2438         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2439         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2440         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
2441         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2442         "n"(int32_t(TransB))); \
2443 }
2444
2445 // ---- TMA Shared Memory Layout ----
2446 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储  $A[BM \times BK] + B[BK \times BN]$ 
2447 //
2448 // 每个 stage 包含两块 smem:
2449 //   A tile:  $[BM, BK]$  row-major  $\rightarrow BK \times BM$  个 half, 地址连续
2450 //   B tile: TMA 按  $[BN, BK]$  物理写入 (详见下方 *),  $BN \times BK$  个 half
2451 //
2452 // * 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
2453 //
2454 //   上层数据: 源矩阵  $B^T [N, K]$  row-major (TN 布局, B 的列以行优先形式存储)。
2455 //   物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为  $[BN, BK]$  row-major
2456 //   (BN=128 行, BK=64 列, 每行 64 个连续的 half)。

```

```

2457 //      TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2458 //      TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2459 //      逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2460 //      通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2461 //      实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2462 //      swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2463 //
2464 //      stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2465 //      128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2466 //      stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2467 //      = 8 rows × (BK=64) halves × 2 bytes = 1024。
2468 //      这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2469 //
2470 //      与 MMA(TN) 的对比:
2471 //      MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2472 //      WGMMMA:    smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2473 //      简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2474 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2475     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2476     // B tile 笔记:
2477     // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2478     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2479     //   以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2480     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN) ,
2481     //   但写 BN×BK 更能体现物理存储形态
2482     alignas(128) half B[BN * BK * QSIZE];
2483 };
2484
2485 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2486 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
2487 //
2488 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2489 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2490 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2491 //
2492 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2493 //   将 A/B tile 从 HBM 搬到 shared memory。
2494 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2495 //
2496 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2497 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2498 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2499 //
2500 // 本节重点理解:
2501 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2502 //   即可搬运整个 2D tile, 无需所有线程参与。
2503 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2504 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2505 //   Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2506 //
2507 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
2508 // WGMMMA Atom:    m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2509 // K Tile:         BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
2510 // M Tile:         BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
2511 // N Tile:         BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
2512 // Block Tile:     C[128,128] = A[128,64] × B[64,128]
2513 // Threads:        256 = 2 warpgroups × 128 threads/warpgroup
2514 //   Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
2515 //   Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
2516 //
2517 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2518 // 的延迟被计算完全掩盖。
2519 //

```

```

2520 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2521 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2522 // Block: (256, 1, 1), 2 warpgroups
2523 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2524 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2525           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2526           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2527           const bool BLOCK_SWIZZLE = false>
2528 __global__ void __launch_bounds__(NUM_THREADS)
2529     hgemm_wgmma_stages_tn(
2530         int M, int N, int K, half *C,
2531         const CUtensorMap *__restrict__ tensorMapA,
2532         const CUtensorMap *__restrict__ tensorMapB) {
2533     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2534     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2535     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2536     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2537     // 不展开宿主侧 create_tensor_map 细节。
2538
2539     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2540     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2541     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2542     const int by = blockIdx.y;
2543     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2544     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2545     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2546     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2547     // 当只有一个 consumer 时, 它负责全部 BM=128 行
2548     constexpr int B_WG_M = BM / num_consumers; // 128
2549
2550     // 边界检查: 确保当前 block 不超出 M/N 范围
2551     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2552         return;
2553
2554     // ---- Shared Memory 分配 ----
2555     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2556     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2557     extern __shared__ __align__(128) uint8_t smem_AB[];
2558     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2559         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
2560     half *s_a = s.A;
2561     half *s_b = s.B;
2562
2563     // ---- cuda::barrier 初始化 ----
2564     //
2565     // ★ Barrier 机制速查 (arrive / wait 核心语义):
2566     //
2567     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
2568     //
2569     //   init(bar, N): 设置 arrive_count = N。
2570     //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2571     //
2572     //   arrive(): 线程声明"我已到达此 barrier 点"。
2573     //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
2574     //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
2575     //
2576     //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
2577     //       - 即: 等待 phase 翻转。
2578     //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
2579     //                       (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
2580     //
2581     // 典型用法: b.wait(b.arrive())
2582     //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)

```

```

2583 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2584 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2585 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
2586 //
2587 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
2588 //
2589 //   Producer (1 thread)           Consumer (128 threads)
2590 //   ────────────                ────────────
2591 //                               C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2592 //   ┌ for each k_tile: ─┐       ┌ for each k_tile: ─┐
2593 //   │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q])
2594 //   │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪
2595 //   │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算
2596 //   │   ↓ 异步拷贝             │ │ C3: wait WGMMMA 完成
2597 //   │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q])
2598 //   │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完
2599 //   └──────────────────┘       └──────────────────┘
2600 //
2601 // 关键不变式 (invariant) :
2602 //   - Producer 不会覆盖 Consumer 正在读的 stage
2603 //   - Consumer 不会读 Producer 还没写完的 stage
2604 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
2605 //
2606 // 每个 stage 有两个 barrier:
2607 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
2608 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
2609 //
2610 // arrive_count = 128 (consumer) + 1 (producer) = 129:
2611 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
2612 #pragma nv_diag_suppress static_var_with_dynamic_init
2613 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
2614 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
2615 #pragma nv_diag_default static_var_with_dynamic_init
2616
2617 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
2618 const int num_blocks_k = K / BK;
2619 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
2620 const int tid = threadIdx.x % 128;   // 0~127 within warpgroup
2621
2622 // 初始化 barriers (仅 thread 0 执行)
2623 if (threadIdx.x == 0) {
2624     for (int i = 0; i < K_STAGE; ++i) {
2625         // arrive_count = 129: 128 consumer + 1 producer
2626         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
2627         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
2628         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
2629         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
2630         init(&empty[i], num_consumers * 128 + 1); // same
2631     }
2632     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
2633     // 对 async proxy (TMA/WGMMMA) 可见。
2634     cuda::device::experimental::fence_proxy_async_shared_cta();
2635 }
2636 __syncthreads();
2637
2638 // =====
2639 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
2640 // =====
2641 //
2642 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
2643 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
2644 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
2645 //

```



```

2646 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
2647 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
2648 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
2649 //
2650 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
2651 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
2652 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
2653 //
2654 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
2655 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
2656 // =====
2657 // Producer Warpgroup (WG0, threadIdx.x 0~127)
2658 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
2659 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
2660 // =====
2661 if (wg_idx == 0) {
2662     if (tid == 0) {
2663         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
2664         int qidx = 0;
2665         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2666             if (qidx == K_STAGE)
2667                 qidx = 0;
2668
2669             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
2670             //
2671             // 模式 empty[qidx].wait(empty[qidx].arrive()):
2672             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
2673             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
2674             //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
2675             //     → phase 立即翻转。
2676             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
2677             //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
2678             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
2679             //
2680             // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有?"
2681             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
2682             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
2683             empty[qidx].wait(empty[qidx].arrive());
2684
2685             // Step P2: 提交 TMA 2D 拷贝指令
2686             // cp_async_bulk_tensor_2d_global_to_shared:
2687             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
2688             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
2689             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
2690             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
2691             //
2692             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
2693             // 无需线程逐元素搬运, 零寄存器开销。
2694             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
2695             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2696                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
2697                 full[qidx]);
2698
2699             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
2700             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2701                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
2702                 full[qidx]);
2703
2704             // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
2705             //
2706             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
2707             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
2708             //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem

```

```

2709 // - Phase 翻转条件:
2710 //      (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
2711 //      (b) 所有声明的 async 字节已写入 smem
2712 //      两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
2713 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
2714 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
2715     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
2716 }
2717 }
2718 }
2719 // =====
2720 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
2721 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
2722 // 所有 128 个线程 (4 warps) 全部参与。
2723 // =====
2724 else {
2725     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
2726     //
2727     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
2728     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
2729     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
2730     //
2731     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
2732     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
2733     for (int i = 0; i < K_STAGE; ++i) {
2734         [[maybe_unused]] auto token = empty[i].arrive();
2735     }
2736
2737     // 累加器寄存器声明
2738     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
2739     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
2740     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
2741     //   - d[*][g][*]: N 方向第 g 组 16 列
2742     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
2743     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
2744     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
2745     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
2746
2747     int qidx = 0;
2748     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
2749     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2750         if (qidx == K_STAGE)
2751             qidx = 0;
2752
2753         // Step C1: 等待 TMA 数据就绪 (full 信号)
2754         //
2755         // 模式 full[qidx].wait(full[qidx].arrive()):
2756         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
2757         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
2758         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
2759         //     贡献第 129 次到达 + TMA 字节全部写完 -> phase 翻转 -> wait 返回。
2760         //
2761         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
2762         full[qidx].wait(full[qidx].arrive());
2763
2764         // Step C2: 发射 WGMMMA 指令序列
2765         //
2766         // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
2767         // 标准流程: FENCE -> 发射 WGMMMA -> COMMIT -> WAIT。
2768         //
2769         // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
2770         //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
2771         //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出

```



```

2772 // - 本质是 proxy 间的内存序 (memory ordering) , 不是传统 __syncthreads
2773 //
2774 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
2775 // D[64×128] += A[64×16] × B[16×128]
2776 // A/B 均为 K-major (imm-trans=0) , 由 descA/descB 描述 smem 中的布局。
2777 // ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都用 ScaleD=1。
2778 WGMMMA_FENCE();
2779
2780 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
2781 // 每个 atom 处理 64 行 M 方向数据。
2782 #pragma unroll
2783 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2784     // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
2785     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
2786     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
2787
2788     // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
2789     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
2790 #pragma unroll
2791     for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
2792         // 第 k_it 次 WGMMMA:
2793         // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
2794         // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
2795         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
2796         // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
2797         // ScaleD=1: 累加 (K 维迭代需要累积结果)
2798         WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
2799                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
2800                                     1, // ScaleD=1: accumulate (不清零)
2801                                     1, 1, 0, 0);
2802     }
2803 }
2804
2805 // Step C3: 提交并等待 WGMMMA 完成
2806 //
2807 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
2808 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
2809 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
2810 //
2811 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
2812 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N) 。N=0 即等待所有 group。
2813 // ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
2814 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
2815 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
2816 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据) 。
2817 WGMMMA_COMMIT_GROUP();
2818 WGMMMA_WAIT_GROUP(0);
2819
2820 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
2821 //
2822 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
2823 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive) 。
2824 //
2825 // 语义: Consumer 说 "stage qidx 我已经读完了, 你可以放心覆盖了。"
2826 [[maybe_unused]] auto token = empty[qidx].arrive();
2827 }
2828
2829 // =====
2830 // Epilogue: 将寄存器中的累加结果写回 global memory C
2831 // =====
2832 //
2833 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2834 //

```

```

2835 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2836 // 这些 half 分布在 d[2][8][4] 数组中:
2837 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2838 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2839 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
2840 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2841 //
2842 // 其中:
2843 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
2844 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2845 //
2846 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2847 // +-----+-----+
2848 // | reg[0] | reg[2] | <- rows [row, row+8)
2849 // |(col,+2)|(col+8)|
2850 // +-----+-----+
2851 // | reg[1] | reg[3] | <- rows [row+8, row+16)
2852 // |(col,+2)|(col+8)|
2853 // +-----+-----+
2854 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2855 //
2856 // 三维遍历:
2857 // m_it=0: rows 0~63, m_it=1: rows 64~127
2858 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2859 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2860 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2861
2862 const int lane = tid % 32;
2863 const int warp = tid / 32;
2864 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2865 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2866 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2867 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2868 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2869 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2870 const int row = warp * 16 + lane / 4;
2871 // block_C: 当前 C tile 在 global memory 中的起始地址
2872 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2873 half *block_C = C + by * BM * N + bx * BN;
2874
2875 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
2876 #pragma unroll
2877 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2878     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2879 #pragma unroll
2880 for (int g = 0; g < WGMMMA_N / 16; ++g) {
2881     int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2882     // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2883     // 左上象限: (row, col)
2884     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2885         d[m_it][g][0];
2886     // 左下象限: (row+8, col)
2887     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2888         d[m_it][g][1];
2889     // 右上象限: (row, col+8)
2890     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2891         d[m_it][g][2];
2892     // 右下象限: (row+8, col+8)
2893     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2894         d[m_it][g][3];
2895 }
2896 }
2897 }

```

```

2898 }
2899
2900 #if (defined(NOTES_V2_ENABLE_WGMMMA))
2901 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
2902 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
2903 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2904 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2905 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2906 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2907 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2908 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2909 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2910 //
2911 // * gmem_prob_stride + 1 的含义:
2912 // 语法层面 — 数组名退化 + 指针算术:
2913 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
2914 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
2915 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
2916 // 语义层面 — 跳过隐式的最内维 stride:
2917 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
2918 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
2919 //   固定等于 sizeof(element_type), 不需要显式传入。
2920 //   对于 tensorRank=2 的 FP16 矩阵:
2921 //     dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
2922 //     dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
2923 //   gmem_prob_stride 数组的布局是内维在前:
2924 //     [0] = sizeof(half)                        ← 内维, 不传
2925 //     [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
2926 //   所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
2927 //
2928 // * smem_box_stride 为什么不需要 +1?
2929 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
2930 //   最内维 stride 也必须显式传入 (不隐式)。
2931 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
2932 //   都是连续存放的, 维度间没有 padding/stride。
2933 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
2934 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
2935 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
2936 //   两个 stride 值, 不需要跳过任何一个。
2937 //
2938 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2939
2940 template <int BlockMajorSize, int BlockMinorSize>
2941 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2942                                               half *gmem_ptr,
2943                                               int blocks_height,
2944                                               int blocks_width) {
2945     void *gmem_address = (void *)gmem_ptr;
2946     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2947                                     (uint64_t)BlockMajorSize * blocks_height,
2948                                     1, 1, 1};
2949     uint64_t gmem_prob_stride[5] = {
2950         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2951     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2952                                   uint32_t(BlockMajorSize), 1, 1, 1};
2953     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2954     CUresult result = cuTensorMapEncodeTiled(
2955         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2956         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2957         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2958         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2959     if (result != CUDA_SUCCESS)
2960         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);

```

```

2961 }
2962
2963 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2964     half *src, int blocks_height, int blocks_width) {
2965     CUTensorMap *tma_map_d;
2966     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2967     CUTensorMap tma_map_host;
2968     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2969     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2970         cudaMemcpyHostToDevice);
2971     return tma_map_d;
2972 }
2973 #endif /* NOTES_V2_ENABLE_WGMMA */
2974
2975 // =====
2976 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2977 // =====
2978 // 面试题点 (FlashAttention 算法) :
2979 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2980 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2981 // 2. FA 三板斧:
2982 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
2983 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2984 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2985 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2986 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2987 //    - 优点: 减少 warp 间通信和 shuffle
2988 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2989 //    for each K,V tile:
2990 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2991 //        m_new = max(m_old, row_max(S_cur * scale))
2992 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2993 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2994 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2995 //        O_final = O_new / l_final
2996 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2997 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
2998 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2999 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3000 //    - 这是此实现的寄存器布局复用技巧, 不要背成 “所有 MMA A/C fragment
3001 //      都天然同构” 的通用结论
3002 //
3003 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3004 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3005 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3006 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3007 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3008
3009 // ---- 寄存器填充辅助函数 ----
3010 template <typename T, int M, const int N, const int K = 2>
3011 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3012     #pragma unroll
3013     for (int i = 0; i < M; ++i)
3014         #pragma unroll
3015         for (int j = 0; j < N; ++j)
3016             #pragma unroll
3017             for (int k = 0; k < K; ++k)
3018                 R[i][j][k] = val;
3019 }
3020
3021 template <typename T, int M, const int N = 2>
3022 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3023     #pragma unroll

```

```

3024     for (int i = 0; i < M; ++i)
3025 #pragma unroll
3026     for (int j = 0; j < N; ++j)
3027         R[i][j] = val;
3028 }
3029
3030 // =====
3031 // FlashAttention-2 Split-Q Kernel (完整实现)
3032 // =====
3033 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3034 //
3035 // Tile 设计 (以 kHeadDim=64 为例) :
3036 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3037 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3038 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3039 //
3040 // 执行流程:
3041 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3042 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3043 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3044 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3045 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3046 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3047 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3048 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3049 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3050 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3051 //   9) Epilogue: warp shuffle + 128-bit collective store
3052
3053 template <
3054     const int kHeadDim,           // head dim: 32, 64, 128
3055     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
3056     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
3057     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
3058     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
3059     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
3060     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3061     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
3062     const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
3063     const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
3064     const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
3065     const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3066     const int kStage,            // pipeline stages for K: 1 or 2
3067     const int kPad>             // padding for bank conflict avoidance
3068 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3069     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3070                                     int QKV_seqlen, int QKV_head) {
3071     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3072     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3073     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3074     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3075     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3076     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3077     const float scale = 1.0f / sqrtf((float)kHeadDim);
3078
3079     // Block indexing
3080     const int QKV_batch_id = blockIdx.y / QKV_head;
3081     const int QKV_head_id = blockIdx.y % QKV_head;
3082     const int Q_tile_id = blockIdx.x;
3083     const int tid = threadIdx.x;
3084     const int warp_id = tid / WARP_SIZE;
3085     const int lane_id = tid % WARP_SIZE;
3086     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段

```

```

3087 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3088
3089 // Global memory base offsets for this (batch, head)
3090 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3091 const int Q_gmem_offset =
3092     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3093     kHeadDim;
3094 const int K_gmem_offset = Q_gmem_offset;
3095 const int V_gmem_offset = Q_gmem_offset;
3096 const int O_gmem_offset = Q_gmem_offset;
3097
3098 // Thread-to-smem mapping for cooperative load
3099 int load_smem_Q_Br = tid / (kNumThreads / Br);
3100 int load_smem_Q_d =
3101     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3102 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3103 int load_smem_K_d =
3104     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3105 int load_smem_V_Bc = tid / (kNumThreads / Bc);
3106 int load_smem_V_d =
3107     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3108
3109 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3110 if (load_gmem_Q_Br >= QKV_seqlen)
3111     return;
3112
3113 // ---- Shared memory layout ----
3114 extern __shared__ half smem[];
3115 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3116 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3117 half *Q_tile_smem = smem;
3118 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3119 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3120 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3121 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3122
3123 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3124 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3125 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3126
3127 // ---- Online Softmax persistent state ----
3128 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3129 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3130 float lane_block_row_max_old[kValTileSeqLenQ][2];
3131 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3132 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3133 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3134
3135 // ---- Register allocation ----
3136 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3137 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3138 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3139 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3140 // 对单个 m16n8k16 tile 而言:
3141 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3142 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3143 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3144 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3145 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3146 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3147     [2]; // O accumulator (final output)
3148
3149 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);

```



```

3150 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3151
3152 // =====
3153 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3154 // =====
3155 {
3156     int load_gmem_Q_addr =
3157         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3158     uint32_t load_smem_Q_ptr =
3159         smem_Q_base_ptr +
3160         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3161 #pragma unroll
3162     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3163         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3164     }
3165     CP_ASYNC_COMMIT_GROUP();
3166 }
3167
3168 // =====
3169 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3170 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3171 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
3172 // =====
3173 if constexpr (kStage > 1) {
3174 #pragma unroll
3175     for (int stage = 0; stage < (kStage - 1); ++stage) {
3176         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3177         int load_gmem_K_addr =
3178             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3179         uint32_t load_smem_K_ptr =
3180             smem_K_base_ptr +
3181             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3182              load_smem_K_d) *
3183              sizeof(half);
3184 #pragma unroll
3185         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3186             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3187         }
3188         CP_ASYNC_COMMIT_GROUP();
3189     }
3190     CP_ASYNC_WAIT_GROUP(kStage - 2);
3191     __syncthreads();
3192 }
3193
3194 // =====
3195 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3196 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3197 // =====
3198 #pragma unroll 1
3199 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3200     int smem_sel = tile_K_seqlen % kStage;
3201     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3202
3203     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3204     if constexpr (kStage > 1) {
3205         // 只有 kStage>1 才能真正做 K 的 pipeline:
3206         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
3207         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3208         // Load current V tile (no pipeline for V — one stage is enough)
3209         {
3210             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3211             int load_gmem_V_addr =
3212                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;

```



```

3213     uint32_t load_smem_V_ptr =
3214         smem_V_base_ptr +
3215         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3216 #pragma unroll
3217     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3218         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3219     }
3220     CP_ASYNC_COMMIT_GROUP();
3221 }
3222
3223 // Prefetch next K tile (pipelined)
3224 if ((tile_K_seqlen + 1) < Tc) {
3225     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3226     int load_gmem_K_addr =
3227         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3228     uint32_t load_smem_K_ptr =
3229         smem_K_base_ptr +
3230         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3231          load_smem_K_d) *
3232         sizeof(half);
3233 #pragma unroll
3234     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3235         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3236     }
3237     CP_ASYNC_COMMIT_GROUP();
3238 }
3239 }
3240
3241 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3242 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3243 #pragma unroll
3244 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3245     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3246     #pragma unroll
3247     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3248         int warp_smem_Q_Br =
3249             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3250         int lane_smem_Q_Br =
3251             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3252         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3253         uint32_t lane_smem_Q_ptr =
3254             smem_Q_base_ptr +
3255             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3256         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3257                    lane_smem_Q_ptr);
3258     }
3259
3260     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3261     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3262     #pragma unroll
3263     for (int j = 0; j < kValTileSeqLenK; ++j) {
3264         int warp_smem_K_Bc =
3265             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3266         int lane_smem_K_Bc =
3267             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3268         int lane_smem_K_d =
3269             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3270         uint32_t lane_smem_K_ptr =
3271             smem_K_base_ptr +
3272             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3273              lane_smem_K_d) *
3274             sizeof(half);
3275         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);

```

```

3276     }
3277
3278     // MMA: S[tile] += Q[tile] @ K^T[tile]
3279 #pragma unroll
3280     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3281 #pragma unroll
3282         for (int j = 0; j < kValTileSeqLenK; ++j) {
3283             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3284                     R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3285                     R_S[i][j][1]);
3286         }
3287     }
3288 } // end loop over d
3289 __syncthreads();
3290
3291 // =====
3292 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3293 // =====
3294 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3295 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3296 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3297 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3298 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3299 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3300 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3301 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3302 // kValTileSeqLenK tiles.
3303
3304 float lane_row_max_new[kValTileSeqLenQ][2];
3305 float lane_row_sum_new[kValTileSeqLenQ][2];
3306 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3307 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3308
3309 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3310 #pragma unroll
3311     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3312 #pragma unroll
3313         for (int j = 0; j < kValTileSeqLenK; ++j) {
3314             // Extract half values from R_S registers (C matrix fragment layout)
3315             float2 t_reg_S_0 =
3316                 __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3317             float2 t_reg_S_1 =
3318                 __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3319             // S = (Q@K^T) * scale
3320             float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3321             float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3322             lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3323             lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3324         }
3325         // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3326         // {0,4,8,...,28} hold valid data)
3327         lane_row_max_new[i][0] =
3328             warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3329         lane_row_max_new[i][1] =
3330             warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3331     }
3332
3333 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3334 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3335 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3336 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3337 #pragma unroll
3338     for (int i = 0; i < kValTileSeqLenQ; ++i) {

```

```

3339 // m_new = max(m_old, m_cur)
3340 float block_row_max_new_0 =
3341     max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3342 float block_row_max_new_1 =
3343     max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3344
3345 #pragma unroll
3346 for (int j = 0; j < kValTileSeqLenK; ++j) {
3347     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3348     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3349
3350     // P = exp(S * scale - m_new), 用 fma 保证精度
3351     t_reg_S_0.x =
3352         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3353     t_reg_S_0.y =
3354         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3355     t_reg_S_1.x =
3356         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3357     t_reg_S_1.y =
3358         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3359
3360     // Accumulate row sums
3361     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3362     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3363
3364     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3365     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3366     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3367 }
3368
3369 // Warp-level reduce sum (warp_size = 4, same as max)
3370 lane_row_sum_new[i][0] =
3371     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3372 lane_row_sum_new[i][1] =
3373     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3374 }
3375 __syncthreads();
3376
3377 // =====
3378 // 3d: P@V - P[Br,Bc] @ V[Bc,d] = 0[Br,d]
3379 // =====
3380 // Wait for V to be ready before computing P@V
3381 if constexpr (kStage > 1) {
3382     if ((tile_K_seqLen + 1) < Tc) {
3383         CP_ASYNC_WAIT_GROUP(1);
3384     } else {
3385         CP_ASYNC_WAIT_GROUP(0);
3386     }
3387 } else {
3388     CP_ASYNC_WAIT_GROUP(0);
3389 }
3390 __syncthreads();
3391
3392 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3393
3394 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3395 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3396 #pragma unroll
3397 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3398     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3399     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3400     // transposed ldmatrix
3401     #pragma unroll

```

```

3402 for (int j = 0; j < kValTileHeadDimV; ++j) {
3403     int warp_smem_V_d =
3404         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3405     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3406     int lane_smem_V_d = warp_smem_V_d;
3407     uint32_t lane_smem_V_ptr =
3408         smem_V_base_ptr +
3409         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3410     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3411 }
3412
3413 // P matrix layout in R_S[i][j][2]:
3414 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3415 //   | 64x64 | warp_KV 0 |
3416 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3417 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3418 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3419 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3420 // tile_V_Bc selects which 16-column slice of P to use:
3421 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3422 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3423 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3424 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3425 // 对应的 MMA A fragment 布局可以这样记:
3426 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3427 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3428 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3429 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3430 // fragment 都无条件成立的通用结论。layout转换逻辑:
3431 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3432 int w = tile_V_Bc * 2;
3433 #pragma unroll
3434 for (int i = 0; i < kValTileSeqLenP; ++i) {
3435     #pragma unroll
3436     for (int j = 0; j < kValTileHeadDimV; ++j) {
3437         HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
3438                 R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3439                 R_V[j][0], R_V[j][1], // B fragment = V
3440                 R_O[i][j][0], R_O[i][j][1]); // C fragment output
3441     }
3442 }
3443 } // end for tile_V_Bc
3444 __syncthreads();
3445
3446 // =====
3447 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
3448 // =====
3449 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 1 的 rescale
3450 #pragma unroll
3451 for (int i = 0; i < kValTileSeqLenP; ++i) {
3452     float block_row_max_new_0 = lane_row_max_new[i][0];
3453     float block_row_max_new_1 = lane_row_max_new[i][1];
3454     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3455     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3456
3457     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3458     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3459
3460     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3461     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3462
3463     // Handle first iteration: m_old = -inf, need to use m_new directly
3464     block_row_max_old_0 =

```

```

3465         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3466     block_row_max_old_1 =
3467         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3468
3469     float rescale_o_factor_0 =
3470         __expf(block_row_max_old_0 - block_row_max_new_0);
3471     float rescale_o_factor_1 =
3472         __expf(block_row_max_old_1 - block_row_max_new_1);
3473
3474     // Rescale O_old + Add P@V in one fused step
3475     #pragma unroll
3476     for (int j = 0; j < kValTileHeadDimV; ++j) {
3477         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3478         //   reg[0] -> rows 0~7 的 {c0,c1}
3479         //   reg[1] -> rows 8~15 的 {c2,c3}
3480         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3481         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3482         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3483         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3484
3485         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
3486         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3487         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3488         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3489         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3490
3491         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3492         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3493     }
3494
3495     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3496     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3497     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3498     lane_block_row_sum_old[i][0] = __fmaf_rn(
3499         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3500     lane_block_row_sum_old[i][1] = __fmaf_rn(
3501         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3502
3503     // Update m
3504     lane_block_row_max_old[i][0] = block_row_max_new_0;
3505     lane_block_row_max_old[i][1] = block_row_max_new_1;
3506 }
3507
3508 // Wait for next K tile to be ready in smem before next iteration
3509 if constexpr (kStage > 1) {
3510     if ((tile_K_seqlen + 1) < Tc) {
3511         CP_ASYNC_WAIT_GROUP(0);
3512     }
3513     __syncthreads();
3514 }
3515 } // end outer loop over K seqlen
3516 __syncthreads();
3517
3518 // =====
3519 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
3520 // =====
3521 #pragma unroll
3522 for (int i = 0; i < kValTileSeqLenP; ++i) {
3523     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3524     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3525     #pragma unroll
3526     for (int j = 0; j < kValTileHeadDimV; ++j) {
3527         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));

```

```

3528     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3529     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3530     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3531     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3532     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3533     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3534     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3535 }
3536 }
3537
3538 // =====
3539 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3540 // =====
3541 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3542 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3543 #pragma unroll
3544 for (int i = 0; i < kValTileSeqLenP; ++i) {
3545 #pragma unroll
3546     for (int j = 0; j < kValTileHeadDimV; ++j) {
3547         uint32_t R_Z[2][4];
3548         R_Z[0][0] = R_D[i][j][0];
3549         R_Z[1][0] = R_D[i][j][1];
3550         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3551         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3552         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3553         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3554         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3555         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3556
3557         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3558         if (lane_id % 4 == 0) {
3559             int store_warp_regs_0_Br =
3560                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3561             int store_lane_gmem_0_Br =
3562                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3563             int store_warp_regs_0_d =
3564                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3565             int store_lane_gmem_0_d = store_warp_regs_0_d;
3566             int store_gmem_0_addr_0 = 0_gmem_offset +
3567                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3568                 store_lane_gmem_0_d;
3569             int store_gmem_0_addr_1 = 0_gmem_offset +
3570                 (store_lane_gmem_0_Br + 8) * kHeadDim +
3571                 store_lane_gmem_0_d;
3572             // LDST128BITS = reinterpret_cast<float4*>
3573             *reinterpret_cast<float4*>(&[store_gmem_0_addr_0]) =
3574                 *reinterpret_cast<float4*>(&R_Z[0][0]);
3575             *reinterpret_cast<float4*>(&[store_gmem_0_addr_1]) =
3576                 *reinterpret_cast<float4*>(&R_Z[1][0]);
3577         }
3578     }
3579 }
3580 }
3581
3582 // =====
3583 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3584 // =====
3585
3586 static inline void check(cudaError_t err, const char *msg) {
3587     if (err != cudaSuccess) {
3588         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3589         exit(EXIT_FAILURE);
3590     }

```



```

3591 }
3592
3593
3594 static void test_block_reduce(int N) {
3595
3596     srand(42);
3597     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3598     for (int i = 0; i < N; i++)
3599         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3600
3601     // CPU reference: sum of all elements
3602     double ref = 0.0;
3603     for (int i = 0; i < N; i++) ref += (double)h_a[i];
3604
3605     float *d_a, *d_y;
3606     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
3607     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
3608
3609     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
3610     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
3611
3612     dim3 block(128);
3613     dim3 grid((N + 127) / 128);
3614     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
3615     check(cudaGetLastError(), "blockreduce launch");
3616     check(cudaDeviceSynchronize(), "blockreduce sync");
3617
3618     float result;
3619     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
3620
3621     float err = fabsf(result - (float)ref);
3622     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
3623         err < 1e-2f ? "PASS" : "FAIL");
3624
3625     free(h_a);
3626     cudaFree(d_a); cudaFree(d_y);
3627 }
3628
3629
3630 static void test_dot(int N) {
3631
3632     srand(42);
3633     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3634     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3635     for (int i = 0; i < N; i++) {
3636         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3637         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3638     }
3639
3640     // CPU reference
3641     double ref = 0.0;
3642     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
3643
3644     float *d_a, *d_b, *d_y;
3645     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
3646     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
3647     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
3648
3649     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
3650     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
3651     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
3652
3653     dim3 block(128);

```

```

3654 dim3 grid((N + 127) / 128);
3655 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
3656 check(cudaGetLastError(), "dot launch");
3657 check(cudaDeviceSynchronize(), "dot sync");
3658
3659 float result;
3660 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
3661
3662 float err = fabsf(result - (float)ref);
3663 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
3664         err < 1e-2f ? "PASS" : "FAIL");
3665
3666 // ---- Dot Vec4 ----
3667 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
3668 dim3 block_v4(32);
3669 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
3670 check(cudaGetLastError(), "dot_vec4 launch");
3671 check(cudaDeviceSynchronize(), "dot_vec4 sync");
3672
3673 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
3674 float err_v4 = fabsf(result - (float)ref);
3675 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
3676
3677 free(h_a); free(h_b);
3678 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
3679 }
3680
3681
3682 static void test_relu(int N) {
3683     srand(42);
3684     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3685     float *h_y = (float *)malloc((size_t)N * sizeof(float));
3686     for (int i = 0; i < N; i++)
3687         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3688
3689     float *d_x, *d_y;
3690     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
3691     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
3692     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
3693
3694     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
3695     dim3 block256(256);
3696     dim3 grid256((N + 255) / 256);
3697     relu<<<grid256, block256>>>(d_x, d_y, N);
3698     check(cudaGetLastError(), "relu launch");
3699     check(cudaDeviceSynchronize(), "relu sync");
3700     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
3701     float max_err = 0.0f;
3702     for (int i = 0; i < N; i++) {
3703         float expected = fmaxf(0.0f, h_x[i]);
3704         float err = fabsf(h_y[i] - expected);
3705         if (err > max_err) max_err = err;
3706     }
3707     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3708
3709     dim3 block64(64);
3710     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
3711     check(cudaGetLastError(), "relu_vec4 launch");
3712     check(cudaDeviceSynchronize(), "relu_vec4 sync");
3713     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
3714     max_err = 0.0f;
3715     for (int i = 0; i < N; i++) {
3716         float expected = fmaxf(0.0f, h_x[i]);

```

```

3717     float err = fabsf(h_y[i] - expected);
3718     if (err > max_err) max_err = err;
3719 }
3720 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3721
3722 free(h_x); free(h_y);
3723 cudaFree(d_x); cudaFree(d_y);
3724 }
3725
3726
3727 static void test_elementwise(int N) {
3728     srand(42);
3729     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3730     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3731     for (int i = 0; i < N; i++) {
3732         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3733         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3734     }
3735
3736     float *d_a, *d_b, *d_c;
3737     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
3738     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
3739     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
3740     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
3741     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
3742
3743     dim3 block256(256);
3744     dim3 grid256((N + 255) / 256);
3745     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
3746     check(cudaGetLastError(), "eadd launch");
3747     check(cudaDeviceSynchronize(), "eadd sync");
3748     float *h_c = (float *)malloc((size_t)N * sizeof(float));
3749     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
3750     float max_err = 0.0f;
3751     for (int i = 0; i < N; i++) {
3752         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3753         if (err > max_err) max_err = err;
3754     }
3755     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3756
3757     dim3 block64(64);
3758     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
3759     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
3760     check(cudaGetLastError(), "eadd_vec4 launch");
3761     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
3762     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
3763     max_err = 0.0f;
3764     for (int i = 0; i < N; i++) {
3765         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3766         if (err > max_err) max_err = err;
3767     }
3768     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3769
3770     free(h_a); free(h_b); free(h_c);
3771     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3772 }
3773
3774
3775 static void test_histogram(int N) {
3776     srand(42);
3777     int BINS = 16;
3778     int *h_hist = (int *)calloc(BINS, sizeof(int));
3779     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));

```

```

3780 int *h_idx = (int *)malloc((size_t)N * sizeof(int));
3781 for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
3782 for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
3783
3784 int *d_idx, *d_hist;
3785 check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
3786 check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
3787 check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
3788 check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
3789
3790 dim3 block256(256);
3791 dim3 grid256((N + 255) / 256);
3792 histogram<<<grid256, block256>>>(d_idx, d_hist, N);
3793 check(cudaGetLastError(), "histogram launch");
3794 check(cudaDeviceSynchronize(), "histogram sync");
3795 check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
3796
3797 float max_err = 0.0f;
3798 for (int i = 0; i < BINS; i++) {
3799     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
3800     if (err > max_err) max_err = err;
3801 }
3802 printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3803
3804 free(h_hist); free(h_hist_ref); free(h_idx);
3805 cudaFree(d_idx); cudaFree(d_hist);
3806 }
3807
3808
3809 static void test_merge_attn_states(int num_tokens, int num_heads,
3810                                   int head_size) {
3811
3812     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
3813     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
3814
3815     float *h_prefix_out = (float *)malloc(out_size);
3816     float *h_suffix_out = (float *)malloc(out_size);
3817     float *h_prefix_lse = (float *)malloc(lse_size);
3818     float *h_suffix_lse = (float *)malloc(lse_size);
3819     float *h_output_ref = (float *)malloc(out_size);
3820
3821     srand(42);
3822     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3823         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3824         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3825     }
3826     for (int i = 0; i < num_heads * num_tokens; i++) {
3827         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3828         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3829     }
3830
3831     // CPU reference: 与 kernel 完全相同的浮点计算
3832     for (int t = 0; t < num_tokens; t++) {
3833         for (int h = 0; h < num_heads; h++) {
3834             float p_lse = h_prefix_lse[h * num_tokens + t];
3835             float s_lse = h_suffix_lse[h * num_tokens + t];
3836             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
3837             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
3838
3839             float max_lse = fmaxf(p_lse, s_lse);
3840             p_lse -= max_lse;
3841             s_lse -= max_lse;
3842             float p_se = expf(p_lse);

```

```

3843     float s_se = expf(s_lse);
3844     float p_scale = p_se / (p_se + s_se);
3845     float s_scale = s_se / (p_se + s_se);
3846
3847     int head_off = t * num_heads * head_size + h * head_size;
3848     for (int d = 0; d < head_size; d++) {
3849         h_output_ref[head_off + d] =
3850             h_prefix_out[head_off + d] * p_scale +
3851             h_suffix_out[head_off + d] * s_scale;
3852     }
3853 }
3854 }
3855
3856 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
3857 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
3858 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
3859 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
3860 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
3861 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
3862
3863 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
3864                 cudaMemcpyHostToDevice),
3865        "merge_attn H2D p_out");
3866 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
3867                 cudaMemcpyHostToDevice),
3868        "merge_attn H2D s_out");
3869 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3870                 cudaMemcpyHostToDevice),
3871        "merge_attn H2D p_lse");
3872 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
3873                 cudaMemcpyHostToDevice),
3874        "merge_attn H2D s_lse");
3875
3876 int threads_per_head = head_size / 4;
3877 int total_threads = num_tokens * num_heads * threads_per_head;
3878 dim3 block(128);
3879 dim3 grid((total_threads + 127) / 128);
3880 merge_attn_states<<<grid, block>>>(&d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3881                                     num_tokens, num_heads, head_size);
3882 check(cudaGetLastError(), "merge_attn launch");
3883 check(cudaDeviceSynchronize(), "merge_attn sync");
3884
3885 float *h_output = (float *)malloc(out_size);
3886 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3887        "merge_attn D2H");
3888
3889 float max_err = 0.0f;
3890 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3891     float err = fabsf(h_output[i] - h_output_ref[i]);
3892     if (err > max_err) max_err = err;
3893 }
3894 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
3895        max_err < 1e-4f ? "PASS" : "FAIL");
3896
3897 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
3898 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
3899 h_suffix_lse[0] = 0.0f;
3900 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3901                 cudaMemcpyHostToDevice),
3902        "merge_attn H2D inf lse");
3903 merge_attn_states<<<grid, block>>>(&d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3904                                     num_tokens, num_heads, head_size);

```

```

3906     num_tokens, num_heads, head_size);
3907     check(cudaGetLastError(), "merge_attn inf launch");
3908     check(cudaDeviceSynchronize(), "merge_attn inf sync");
3909     check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3910           "merge_attn inf D2H");
3911
3912     // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
3913     float inf_err = 0.0f;
3914     for (int d = 0; d < head_size; d++) {
3915         float err = fabsf(h_output[d] - h_suffix_out[d]);
3916         if (err > inf_err) inf_err = err;
3917     }
3918     printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
3919           inf_err < 1e-4f ? "PASS" : "FAIL");
3920
3921     free(h_prefix_out);
3922     free(h_suffix_out);
3923     free(h_prefix_lse);
3924     free(h_suffix_lse);
3925     free(h_output_ref);
3926     free(h_output);
3927     cudaFree(d_prefix_out);
3928     cudaFree(d_suffix_out);
3929     cudaFree(d_prefix_lse);
3930     cudaFree(d_suffix_lse);
3931     cudaFree(d_output);
3932 }
3933
3934
3935 static void test_softmax(int N) {
3936     // Use N = blockDim.x so one block processes all elements as one token.
3937     constexpr int NUM_THREADS = 256;
3938     if (N != NUM_THREADS) {
3939         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
3940         return;
3941     }
3942
3943     srand(42);
3944     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3945     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
3946     for (int i = 0; i < N; i++)
3947         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
3948
3949     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
3950     float max_val = -FLT_MAX;
3951     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
3952     double sum_exp = 0.0;
3953     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
3954     for (int i = 0; i < N; i++)
3955         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
3956
3957     float *d_x, *d_y;
3958     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
3959     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
3960
3961     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
3962
3963     dim3 block(256);
3964     dim3 grid(1); // one token covering all N elements
3965     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3966     check(cudaGetLastError(), "softmax launch");
3967     check(cudaDeviceSynchronize(), "softmax sync");
3968

```

```

3969 float *h_y = (float *)malloc((size_t)N * sizeof(float));
3970 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
3971
3972 float max_err = 0.0f;
3973 for (int i = 0; i < N; i++) {
3974     float err = fabsf(h_y[i] - h_y_ref[i]);
3975     if (err > max_err) max_err = err;
3976 }
3977 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3978
3979 // ---- Safe Softmax ----
3980 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3981 check(cudaGetLastError(), "safe_softmax launch");
3982 check(cudaDeviceSynchronize(), "safe_softmax sync");
3983 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
3984 max_err = 0.0f;
3985 for (int i = 0; i < N; i++) {
3986     float err = fabsf(h_y[i] - h_y_ref[i]);
3987     if (err > max_err) max_err = err;
3988 }
3989 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3990
3991 // ---- Naive Softmax ----
3992 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3993 check(cudaGetLastError(), "naive_softmax launch");
3994 check(cudaDeviceSynchronize(), "naive_softmax sync");
3995 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
3996 max_err = 0.0f;
3997 for (int i = 0; i < N; i++) {
3998     float err = fabsf(h_y[i] - h_y_ref[i]);
3999     if (err > max_err) max_err = err;
4000 }
4001 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4002
4003 free(h_x); free(h_y); free(h_y_ref);
4004 cudaFree(d_x); cudaFree(d_y);
4005 }
4006
4007
4008 static void test_rms_norm(int N, int K) {
4009
4010     srand(42);
4011     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4012     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4013     for (int i = 0; i < N * K; i++)
4014         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4015     float g = 1.5f; // gain
4016
4017     // CPU reference: y = (x / rms(x)) * g
4018     float epsilon = 1e-5f;
4019     for (int n = 0; n < N; n++) {
4020         double sum_sq = 0.0;
4021         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4022         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4023         for (int k = 0; k < K; k++)
4024             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4025     }
4026
4027     float *d_x, *d_y;
4028     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4029     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4030     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4031

```



```

4032 dim3 block(128);
4033 dim3 grid(N);
4034 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4035 check(cudaGetLastError(), "rmsnorm launch");
4036 check(cudaDeviceSynchronize(), "rmsnorm sync");
4037
4038 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4039 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4040
4041 float max_err = 0.0f;
4042 for (int i = 0; i < N * K; i++) {
4043     float err = fabsf(h_y[i] - h_y_ref[i]);
4044     if (err > max_err) max_err = err;
4045 }
4046 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4047
4048 // ---- RMS Norm Vec4 ----
4049 dim3 block_rv4(32);
4050 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4051 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4052 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4053 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4054 max_err = 0.0f;
4055 for (int i = 0; i < N * K; i++) {
4056     float err = fabsf(h_y[i] - h_y_ref[i]);
4057     if (err > max_err) max_err = err;
4058 }
4059 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4060
4061 free(h_x); free(h_y); free(h_y_ref);
4062 cudaFree(d_x); cudaFree(d_y);
4063 }
4064
4065
4066 static void test_layer_norm(int N, int K) {
4067
4068     srand(42);
4069     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4070     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4071     for (int i = 0; i < N * K; i++)
4072         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4073     float g = 1.5f, b = 0.3f; // gain and bias
4074
4075     // CPU reference: y = ((x - mean) / std) * g + b
4076     float epsilon = 1e-5f;
4077     for (int n = 0; n < N; n++) {
4078         double sum = 0.0;
4079         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4080         float mean = (float)sum / (float)K;
4081         double sum_sq = 0.0;
4082         for (int k = 0; k < K; k++) {
4083             float diff = h_x[n * K + k] - mean;
4084             sum_sq += (double)diff * (double)diff;
4085         }
4086         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4087         for (int k = 0; k < K; k++)
4088             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4089     }
4090
4091     float *d_x, *d_y;
4092     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4093     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4094     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");

```

```

4095 dim3 block(128);
4096 dim3 grid(N);
4097 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4098 check(cudaGetLastError(), "layernorm launch");
4099 check(cudaDeviceSynchronize(), "layernorm sync");
4100
4101 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4102 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4103
4104 float max_err = 0.0f;
4105 for (int i = 0; i < N * K; i++) {
4106     float err = fabsf(h_y[i] - h_y_ref[i]);
4107     if (err > max_err) max_err = err;
4108 }
4109 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4110
4111 // ---- Layer Norm Vec4 ----
4112 dim3 block_lv4(32);
4113 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4114 check(cudaGetLastError(), "layernorm_vec4 launch");
4115 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4116 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4117 max_err = 0.0f;
4118 for (int i = 0; i < N * K; i++) {
4119     float err = fabsf(h_y[i] - h_y_ref[i]);
4120     if (err > max_err) max_err = err;
4121 }
4122 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4123
4124 free(h_x); free(h_y); free(h_y_ref);
4125 cudaFree(d_x); cudaFree(d_y);
4126 }
4127
4128
4129 static void test_rope(int seq_len, int N) {
4130
4131     int total_pairs = seq_len * N;
4132     int total_elems = total_pairs * 2;
4133     size_t size = (size_t)total_elems * sizeof(float);
4134
4135     float *h_x = (float *)malloc(size);
4136     float *h_y_ref = (float *)malloc(size);
4137
4138     srand(42);
4139     for (int i = 0; i < total_elems; i++)
4140         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4141
4142     // CPU reference: 2D rotation for each pair
4143     for (int idx = 0; idx < total_pairs; idx++) {
4144         int token_pos = idx / N;
4145         int token_idx = idx % N;
4146         float x1 = h_x[idx * 2];
4147         float x2 = h_x[idx * 2 + 1];
4148         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4149         float angle = (float)token_pos * theta;
4150         float cos_v = cosf(angle);
4151         float sin_v = sinf(angle);
4152         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4153         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4154     }
4155
4156     float *d_x, *d_y;

```

```

4158 check(cudaMalloc(&d_x, size), "rope alloc X");
4159 check(cudaMalloc(&d_y, size), "rope alloc Y");
4160 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4161
4162 dim3 block(256);
4163 dim3 grid((total_pairs + 255) / 256);
4164 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4165 check(cudaGetLastError(), "rope launch");
4166 check(cudaDeviceSynchronize(), "rope sync");
4167
4168 float *h_y = (float *)malloc(size);
4169 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4170
4171 float max_err = 0.0f;
4172 for (int i = 0; i < total_elems; i++) {
4173     float err = fabsf(h_y[i] - h_y_ref[i]);
4174     if (err > max_err) max_err = err;
4175 }
4176 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4177
4178 free(h_x);
4179 free(h_y);
4180 free(h_y_ref);
4181 cudaFree(d_x);
4182 cudaFree(d_y);
4183 }
4184
4185
4186 static void test_mat_transpose(int row, int col) {
4187
4188     size_t size_in = (size_t)row * col * sizeof(float);
4189     size_t size_out = (size_t)col * row * sizeof(float);
4190
4191     float *h_x = (float *)malloc(size_in);
4192     float *h_y_ref = (float *)malloc(size_out);
4193
4194     srand(42);
4195     for (int i = 0; i < row * col; i++)
4196         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4197
4198     // CPU reference: y[j][i] = x[i][j] (row-major)
4199     for (int i = 0; i < row; i++)
4200         for (int j = 0; j < col; j++)
4201             h_y_ref[j * row + i] = h_x[i * col + j];
4202
4203     float *d_x, *d_y;
4204     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4205     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4206     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4207
4208     dim3 block(16, 16);
4209     dim3 grid((col + 15) / 16, (row + 15) / 16);
4210     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4211     check(cudaGetLastError(), "mattrans launch");
4212     check(cudaDeviceSynchronize(), "mattrans sync");
4213
4214     float *h_y = (float *)malloc(size_out);
4215     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4216
4217     float max_err = 0.0f;
4218     for (int i = 0; i < col * row; i++) {
4219         float err = fabsf(h_y[i] - h_y_ref[i]);
4220         if (err > max_err) max_err = err;

```

```

4221 }
4222 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4223
4224 free(h_x);
4225 free(h_y);
4226 free(h_y_ref);
4227 cudaFree(d_x);
4228 cudaFree(d_y);
4229 }
4230
4231
4232 static void test_mat_transpose_padded(int row, int col) {
4233
4234     size_t size_in = (size_t)row * col * sizeof(float);
4235     size_t size_out = (size_t)col * row * sizeof(float);
4236
4237     float *h_x = (float *)malloc(size_in);
4238     float *h_y_ref = (float *)malloc(size_out);
4239
4240     srand(42);
4241     for (int i = 0; i < row * col; i++)
4242         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4243
4244     // CPU reference: y[j][i] = x[i][j] (row-major)
4245     for (int i = 0; i < row; i++)
4246         for (int j = 0; j < col; j++)
4247             h_y_ref[j * row + i] = h_x[i * col + j];
4248
4249     float *d_x, *d_y;
4250     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4251     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4252     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4253
4254     dim3 block(16, 16);
4255     dim3 grid((col + 15) / 16, (row + 63) / 64);
4256     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4257     check(cudaGetLastError(), "mattrans_padded launch");
4258     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4259
4260     float *h_y = (float *)malloc(size_out);
4261     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4262
4263     float max_err = 0.0f;
4264     for (int i = 0; i < col * row; i++) {
4265         float err = fabsf(h_y[i] - h_y_ref[i]);
4266         if (err > max_err) max_err = err;
4267     }
4268     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4269
4270     free(h_x);
4271     free(h_y);
4272     free(h_y_ref);
4273     cudaFree(d_x);
4274     cudaFree(d_y);
4275 }
4276
4277
4278 static void test_sgemv(int M, int K) {
4279
4280     srand(42);
4281     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4282     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4283     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));

```

```

4284 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4285 for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4286
4287 // CPU reference: y[m] = sum_k A[m,k] * x[k]
4288 for (int m = 0; m < M; m++) {
4289     double sum = 0.0;
4290     for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4291     h_y_ref[m] = (float)sum;
4292 }
4293
4294 float *d_a, *d_x, *d_y;
4295 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4296 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4297 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4298
4299 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4300 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4301
4302 dim3 block(32, 4);
4303 dim3 grid((M + 3) / 4);
4304 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4305 check(cudaGetLastError(), "sgemv launch");
4306 check(cudaDeviceSynchronize(), "sgemv sync");
4307
4308 float *h_y = (float *)malloc((size_t)M * sizeof(float));
4309 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4310
4311 float max_err = 0.0f;
4312 for (int m = 0; m < M; m++) {
4313     float err = fabsf(h_y[m] - h_y_ref[m]);
4314     if (err > max_err) max_err = err;
4315 }
4316 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4317
4318 // ---- SGEMV K32 ----
4319 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4320 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4321 check(cudaGetLastError(), "sgemv_k32 launch");
4322 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4323
4324 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4325
4326 max_err = 0.0f;
4327 for (int m = 0; m < M; m++) {
4328     float err = fabsf(h_y[m] - h_y_ref[m]);
4329     if (err > max_err) max_err = err;
4330 }
4331 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4332
4333 // ---- SGEMV K16 ----
4334 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4335 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4336
4337 int K16 = 16;
4338 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4339 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4340 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4341 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4342 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4343
4344 for (int m = 0; m < M; m++) {
4345     double sum = 0.0;
4346     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];

```

```

4347     h_y_ref[m] = (float)sum;
4348 }
4349
4350 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4351 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4352 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4353
4354 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4355 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4356
4357 dim3 grid_k16((M + 7) / 8);
4358 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4359 check(cudaGetLastError(), "sgemv_k16 launch");
4360 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4361
4362 h_y = (float *)malloc((size_t)M * sizeof(float));
4363 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4364
4365 max_err = 0.0f;
4366 for (int m = 0; m < M; m++) {
4367     float err = fabsf(h_y[m] - h_y_ref[m]);
4368     if (err > max_err) max_err = err;
4369 }
4370 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4371
4372 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4373 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4374 }
4375
4376
4377 static void test_sgemm(int M, int N, int K) {
4378
4379     size_t size_a = (size_t)M * K * sizeof(float);
4380     size_t size_b = (size_t)K * N * sizeof(float);
4381     size_t size_c = (size_t)M * N * sizeof(float);
4382
4383     float *h_a = (float *)malloc(size_a);
4384     float *h_b = (float *)malloc(size_b);
4385     float *h_c_ref = (float *)malloc(size_c);
4386
4387     srand(42);
4388     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4389     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4390
4391     float *d_a, *d_b, *d_c;
4392     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4393     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4394     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4395
4396     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4397     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4398
4399     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4400     cublasHandle_t handle;
4401     cublasCreate(&handle);
4402     float alpha = 1.0f, beta = 0.0f;
4403     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4404                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
4405     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4406
4407     // Kernel
4408     cudaEvent_t start, stop;
4409     cudaEventCreate(&start);

```

```

4410 cudaEventCreate(&stop);
4411
4412 dim3 block(32, 32);
4413 dim3 grid((N + 31) / 32, (M + 31) / 32);
4414 sgemmm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4415 check(cudaGetLastError(), "sgemm launch");
4416 check(cudaDeviceSynchronize(), "sgemm sync");
4417
4418 float *h_c = (float *)malloc(size_c);
4419 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4420
4421 // Verify
4422 float max_err = 0.0f;
4423 for (int i = 0; i < M * N; i++) {
4424     float err = fabsf(h_c[i] - h_c_ref[i]);
4425     if (err > max_err) max_err = err;
4426 }
4427 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4428
4429 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4430
4431 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4432 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4433 sgemmm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4434 check(cudaGetLastError(), "sgemm_vec4 launch");
4435 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4436
4437 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4438
4439 max_err = 0.0f;
4440 for (int i = 0; i < M * N; i++) {
4441     float err = fabsf(h_c[i] - h_c_ref[i]);
4442     if (err > max_err) max_err = err;
4443 }
4444 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4445
4446 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4447 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4448 cublasDestroy(handle);
4449 cudaEventDestroy(start);
4450 cudaEventDestroy(stop);
4451 }
4452
4453
4454 static void test_hgemmm_mma(int M, int N, int K) {
4455
4456     size_t size_a = (size_t)M * K * sizeof(half);
4457     size_t size_b = (size_t)K * N * sizeof(half);
4458     size_t size_c = (size_t)M * N * sizeof(half);
4459
4460     half *h_a = (half *)malloc(size_a);
4461     half *h_b = (half *)malloc(size_b);
4462     half *h_c_ref = (half *)malloc(size_c);
4463
4464     srand(42);
4465     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4466     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4467
4468     // Kernel expects B^T [NxK] row-major layout (TN layout convention).
4469     // We store h_b as B [KxN] row-major for cuBLAS, then create B^T for kernel.
4470     size_t size_b_t = (size_t)N * K * sizeof(half);
4471     half *h_b_t = (half *)malloc(size_b_t);
4472     for (int n = 0; n < N; n++)

```



```

4473     for (int k = 0; k < K; k++)
4474         h_b_t[n * K + k] = h_b[k * N + n];
4475
4476     half *d_a, *d_b, *d_b_t, *d_c;
4477     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4478     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4479     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4480     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
4481
4482     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4483     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4484     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4485
4486     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4487     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4488     cublasHandle_t handle;
4489     cublasCreate(&handle);
4490     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4491     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4492                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4493                 &beta_h, d_c, CUDA_R_16F, N,
4494                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4495     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4496
4497     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4498     cudaEvent_t start, stop;
4499     cudaEventCreate(&start);
4500     cudaEventCreate(&stop);
4501
4502     constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4503     size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4504     dim3 block(256);
4505     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4506     hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4507     check(cudaGetLastError(), "hgemm launch");
4508     check(cudaDeviceSynchronize(), "hgemm sync");
4509
4510     half *h_c = (half *)malloc(size_c);
4511     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4512
4513     // Verify
4514     float max_err = 0.0f;
4515     for (int i = 0; i < M * N; i++) {
4516         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4517         if (err > max_err) max_err = err;
4518     }
4519     printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4520
4521     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4522     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4523     cublasDestroy(handle);
4524     cudaEventDestroy(start);
4525     cudaEventDestroy(stop);
4526 }
4527
4528
4529 static void test_hgemm_swizzle(int M, int N, int K) {
4530     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4531     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
4532     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4533
4534     size_t size_a = (size_t)M * K * sizeof(half);
4535     size_t size_b = (size_t)K * N * sizeof(half);

```

```

4536 size_t size_c = (size_t)M * N * sizeof(half);
4537
4538 half *h_a = (half *)malloc(size_a);
4539 half *h_b = (half *)malloc(size_b);
4540 half *h_c_ref = (half *)malloc(size_c);
4541
4542 srand(42);
4543 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4544 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4545
4546 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4547 size_t size_b_t = (size_t)N * K * sizeof(half);
4548 half *h_b_t = (half *)malloc(size_b_t);
4549 for (int n = 0; n < N; n++)
4550     for (int k = 0; k < K; k++)
4551         h_b_t[n * K + k] = h_b[k * N + n];
4552
4553 half *d_a, *d_b, *d_b_t, *d_c;
4554 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4555 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4556 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4557 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4558
4559 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4560 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4561 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4562
4563 // cuBLAS FP16 reference
4564 cublasHandle_t handle;
4565 cublasCreate(&handle);
4566 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4567 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4568             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4569             &beta_h, d_c, CUDA_R_16F, N,
4570             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4571 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4572
4573 // MMA swizzle kernel (default params: K_STAGE=3)
4574 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE_S = 3;
4575 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4576 dim3 block(256);
4577 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4578 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4579 check(cudaGetLastError(), "hgemm_swizzle launch");
4580 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4581
4582 half *h_c = (half *)malloc(size_c);
4583 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4584
4585 // Verify
4586 float max_err = 0.0f;
4587 for (int i = 0; i < M * N; i++) {
4588     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4589     if (err > max_err) max_err = err;
4590 }
4591 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA Swizzle", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4592
4593 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4594 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4595 cublasDestroy(handle);
4596 }
4597
4598

```

```

4599 #if defined(NOTES_V2_ENABLE_CUTE)
4600 static void test_hgemmm_cute(int M, int N, int K) {
4601     // HGEMM CuTe — SM80_16x8x16_F16F16F16_TN + Swizzle<3,3,3> + kStage=2
4602     // TN layout: C[MxN] = A[MxK] × BT[N×K]
4603     // Kernel: hgemmm_mma_stages_tn_cute via launch_hgemmm_mma_stages_tn_cute
4604     // Tile: BM=128, BN=256, BK=32, 128 threads/block
4605
4606     size_t size_a = (size_t)M * K * sizeof(half);
4607     size_t size_b = (size_t)K * N * sizeof(half);
4608     size_t size_c = (size_t)M * N * sizeof(half);
4609
4610     half *h_a = (half *)malloc(size_a);
4611     half *h_b = (half *)malloc(size_b);
4612     half *h_c_ref = (half *)malloc(size_c);
4613
4614     srand(42);
4615     for (int i = 0; i < M * K; i++)
4616         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4617     for (int i = 0; i < K * N; i++)
4618         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4619
4620     // CuTe kernel expects BT [N×K] row-major (TN layout).
4621     size_t size_b_t = (size_t)N * K * sizeof(half);
4622     half *h_b_t = (half *)malloc(size_b_t);
4623     for (int n = 0; n < N; n++)
4624         for (int k = 0; k < K; k++)
4625             h_b_t[n * K + k] = h_b[k * N + n];
4626
4627     half *d_a, *d_b, *d_b_t, *d_c;
4628     check(cudaMalloc(&d_a, size_a), "hgemmm_cute alloc A");
4629     check(cudaMalloc(&d_b, size_b), "hgemmm_cute alloc B (cuBLAS)");
4630     check(cudaMalloc(&d_b_t, size_b_t), "hgemmm_cute alloc B_t (kernel)");
4631     check(cudaMalloc(&d_c, size_c), "hgemmm_cute alloc C");
4632
4633     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemmm_cute H2D A");
4634     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemmm_cute H2D B (cuBLAS)");
4635     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemmm_cute H2D B_t (kernel)");
4636
4637     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4638     cublasHandle_t handle;
4639     cublasCreate(&handle);
4640     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4641     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4642                  CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4643                  CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4644     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm_cute D2H ref");
4645
4646     // CuTe kernel (Stages=2, TN layout)
4647     launch_hgemmm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
4648     check(cudaGetLastError(), "hgemmm_cute launch");
4649     check(cudaDeviceSynchronize(), "hgemmm_cute sync");
4650
4651     half *h_c = (half *)malloc(size_c);
4652     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm_cute D2H");
4653
4654     // Verify
4655     float max_err = 0.0f;
4656     for (int i = 0; i < M * N; i++) {
4657         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4658         if (err > max_err) max_err = err;
4659     }
4660     printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4661

```

```

4662 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4663 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4664 cublasDestroy(handle);
4665 }
4666 #endif /* NOTES_V2_ENABLE_CUTE */
4667
4668
4669 #if defined(NOTES_V2_ENABLE_WGMMMA)
4670 static void test_hgemm_wgmma(int M, int N, int K) {
4671     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
4672     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4673     // Kernel: hgemm_wgmma_stages_tn with default template params
4674
4675     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
4676
4677     // M, K must be divisible by tile dims
4678     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
4679         printf("| %-35s | %-12s | %-4s |\n",
4680             "HGEMM WGMMMA", "SKIP", "SKIP");
4681         return;
4682     }
4683
4684     size_t size_a = (size_t)M * K * sizeof(half);
4685     size_t size_b = (size_t)K * N * sizeof(half);
4686     size_t size_c = (size_t)M * N * sizeof(half);
4687
4688     half *h_a = (half *)malloc(size_a);
4689     half *h_b = (half *)malloc(size_b);
4690     half *h_c_ref = (half *)malloc(size_c);
4691
4692     srand(42);
4693     for (int i = 0; i < M * K; i++)
4694         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4695     for (int i = 0; i < K * N; i++)
4696         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4697
4698     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
4699     size_t size_b_t = (size_t)N * K * sizeof(half);
4700     half *h_b_t = (half *)malloc(size_b_t);
4701     for (int n = 0; n < N; n++)
4702         for (int k = 0; k < K; k++)
4703             h_b_t[n * K + k] = h_b[k * N + n];
4704
4705     half *d_a, *d_b, *d_b_t, *d_c;
4706     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
4707     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
4708     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
4709     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
4710
4711     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
4712     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
4713         "wgmma H2D B (cuBLAS)");
4714     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
4715         "wgmma H2D B_t (kernel)");
4716
4717     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
4718     cublasHandle_t handle;
4719     cublasCreate(&handle);
4720     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4721     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4722         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4723         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4724     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),

```

```

4725     "wgmma D2H ref");
4726
4727 // Create TMA tensor maps for A and B^T
4728 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
4729 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
4730 CUTensorMap *tma_a =
4731     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
4732 CUTensorMap *tma_b =
4733     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
4734
4735 // Launch WGMMMA kernel
4736 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
4737 size_t smem_bytes =
4738     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
4739 cudaFuncSetAttribute(
4740     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
4741     false>,
4742     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
4743
4744 dim3 block(NUM_THREADS);
4745 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4746 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
4747     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
4748 check(cudaGetLastError(), "wgmma launch");
4749 check(cudaDeviceSynchronize(), "wgmma sync");
4750
4751 half *h_c = (half *)malloc(size_c);
4752 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
4753
4754 // Verify
4755 float max_err = 0.0f;
4756 for (int i = 0; i < M * N; i++) {
4757     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4758     if (err > max_err)
4759         max_err = err;
4760 }
4761 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
4762     max_err < 1.0f ? "PASS" : "FAIL");
4763
4764 free(h_a);
4765 free(h_b);
4766 free(h_b_t);
4767 free(h_c);
4768 free(h_c_ref);
4769 cudaFree(d_a);
4770 cudaFree(d_b);
4771 cudaFree(d_b_t);
4772 cudaFree(d_c);
4773 cudaFree(tma_a);
4774 cudaFree(tma_b);
4775 cublasDestroy(handle);
4776 }
4777 #endif /* NOTES_V2_ENABLE_WGMMMA */
4778
4779
4780 static void test_flash_attn(int seqlen, int head_dim) {
4781     // FlashAttention-2 with split-Q, MMA m16n8k16
4782     int B = 1, H = 8;
4783
4784     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
4785
4786     srand(42);
4787     half *h_q = (half *)malloc(sz);

```

```

4788 half *h_k = (half *)malloc(sz);
4789 half *h_v = (half *)malloc(sz);
4790 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
4791     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4792     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4793     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4794 }
4795
4796 // CPU reference in FP32: O = softmax(Q @ K^T / sqrt(d)) @ V
4797 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
4798 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
4799 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
4800 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
4801 int count = B * H * seqlen * head_dim;
4802 for (int i = 0; i < count; i++) {
4803     ref_q[i] = __half2float(h_q[i]);
4804     ref_k[i] = __half2float(h_k[i]);
4805     ref_v[i] = __half2float(h_v[i]);
4806 }
4807
4808 float scale = 1.0f / sqrtf((float)head_dim);
4809 for (int bi = 0; bi < B * H; bi++) {
4810     for (int qi = 0; qi < seqlen; qi++) {
4811         // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
4812         float smax = -INFINITY;
4813         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
4814         for (int kj = 0; kj < seqlen; kj++) {
4815             float s = 0.0f;
4816             for (int d = 0; d < head_dim; d++)
4817                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
4818                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
4819             S[kj] = s * scale;
4820             if (S[kj] > smax) smax = S[kj];
4821         }
4822         // softmax
4823         double sum_exp = 0.0;
4824         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
4825         float inv_sum = 1.0f / (float)sum_exp;
4826         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
4827         for (int d = 0; d < head_dim; d++) {
4828             double o_acc = 0.0;
4829             for (int kj = 0; kj < seqlen; kj++)
4830                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
4831                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
4832             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
4833         }
4834         free(S);
4835     }
4836 }
4837
4838 half *d_q, *d_k, *d_v, *d_o;
4839 check(cudaMalloc(&d_q, sz), "fa alloc Q");
4840 check(cudaMalloc(&d_k, sz), "fa alloc K");
4841 check(cudaMalloc(&d_v, sz), "fa alloc V");
4842 check(cudaMalloc(&d_o, sz), "fa alloc O");
4843 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
4844 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
4845 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
4846
4847 // Template params for kHeadDim=64, kStage=2
4848 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
4849 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
4850 constexpr int kMmaTileSeqLenQ = 4;

```

```

4851 constexpr int kMmaTileSeqLenK = 1;
4852 constexpr int kMmaTileSeqLenP = 4;
4853 constexpr int kMmaTileHeadDimV = 1;
4854 constexpr int kValTileSeqLenQ = 1;
4855 constexpr int kValTileSeqLenK = 8;
4856 constexpr int kValTileSeqLenP = 1;
4857 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
4858
4859 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
4860 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
4861 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
4862                     kStageV * Bc * (kHeadDim + kPadV) +
4863                     Bc * (kHeadDim + kPadV)) * sizeof(half);
4864
4865 dim3 block(128);
4866 dim3 grid((seqlen + Br - 1) / Br, B * H);
4867
4868 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
4869                               kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
4870                               kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
4871                               kStageV, kPadV>
4872     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
4873 check(cudaGetLastError(), "fa launch");
4874 check(cudaDeviceSynchronize(), "fa sync");
4875
4876 half *h_o = (half *)malloc(sz);
4877 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
4878
4879 float max_err = 0.0f;
4880 for (int i = 0; i < count; i++) {
4881     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
4882     if (err > max_err) max_err = err;
4883 }
4884 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
4885
4886 free(h_q); free(h_k); free(h_v); free(h_o);
4887 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
4888 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
4889 }
4890
4891
4892 int main(int argc, char *argv[]) {
4893 #if defined(NOTES_V2_ENABLE_WGMMMA)
4894     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
4895 #endif
4896     int M = 1024, N = 1024, K = 1024;
4897     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
4898
4899     printf("=== notes-v2.cu verification harness ===\n");
4900     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
4901     printf("| -----|-----|-----|\n");
4902
4903     test_block_reduce(N);
4904     test_dot(N);
4905     test_relu(1024);
4906     test_elementwise(1024);
4907     test_histogram(1024);
4908     test_merge_attn_states(512, 16, 128);
4909     test_softmax(256);
4910     test_rms_norm(8, 128);
4911     test_layer_norm(8, 128);
4912     test_rope(8, 128);
4913     test_mat_transpose(256, 256);

```



```

4914     test_mat_transpose_padded(256, 256);
4915     test_sgemv(256, 128);
4916     test_sgemm(M, N, K);
4917     test_hgemm_mma(M, N, K);
4918     test_hgemm_swizzle(M, N, K);
4919     #if (defined(NOTES_V2_ENABLE_CUTE))
4920         test_hgemm_cute(M, N, K);
4921     #endif
4922     #if (defined(NOTES_V2_ENABLE_WGMMA))
4923         test_hgemm_wgmma(M, N, K);
4924     #endif
4925     test_flash_attn(1024, 64);
4926
4927     printf("=== All tests done ===\n");
4928     return 0;
4929 }
4930
4931 // =====
4932 // Quick build & run reference
4933 // =====
4934 // # sm_89 单独编译 + 运行:
4935 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
4936 //
4937 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
4938 //   项目内置 third-party/cutlass) :
4939 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
4940 //   -I ../../third-party/cutlass/include \
4941 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
4942 //
4943 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
4944 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4945 //   -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
4946 //
4947 // # sm_90a + CuTe + WGMMA:
4948 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4949 //   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
4950 //   -I ../../third-party/cutlass/include \
4951 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```